EXPERIMENT-1

1. a) Install Flutter and Dart SDK.

b) Write a simple Dart program to understand the language basics.

a) Install Flutter and Dart SDK.

What is Flutter?

Flutter is an open-source **UI software development toolkit** created by **Google**. It is used to build **natively compiled** applications for:

- **Mobile** platforms (Android & iOS),
- **Web** browsers, and
- **Desktop** operating systems (Windows, macOS, Linux)

… all from a **single codebase**.

Flutter uses a rich collection of **widgets** to design responsive and flexible user interfaces. Because Flutter compiles directly to native machine code, it offers high performance and smooth animations across all supported platforms.

What is Dart Programming Language?

Dart is a programming language developed by **Google**, primarily designed to build applications with Flutter.

- It is **object-oriented**, **strongly typed**, and supports **both just-in-time (JIT)** and **ahead-of-time (AOT)** compilation.
- Dart code can be compiled to **native machine code** for mobile and desktop apps or to **JavaScript** for web applications.
- Dart is the primary language used to write Flutter applications due to its simplicity and performance advantages.

Uses of Flutter and Dart

Flutter is used for:

- Developing **cross-platform applications** with native-like performance and appearance.
- Creating rich and highly customizable **user interfaces** using a flexible widget system.
- Rapid development cycles through **hot reload**, which lets developers see changes in real-time without restarting the app.
- Building applications for **Android**, **iOS**, **web**, and **desktop platforms** from a single codebase, significantly reducing development time and effort.

Dart is used for:

- Writing **Flutter applications**.
- Backend and server-side development.
- Creating **command-line tools**.
- Developing web applications by compiling Dart to JavaScript.

Step-by-Step Installation of Flutter and Dart

Note: Flutter comes bundled with the Dart SDK. You do **not** need to install Dart separately when installing Flutter.

Step 1: System Requirements

For Windows:

- Operating System: Windows 10 or 11 (64-bit)
- Available Disk Space: At least 1.64 GB (excluding IDEs and other tools)
- Additional Tools: Git installed, PowerShell available, and Chrome (required for Flutter web development)

Step 2: Download Flutter SDK

1. Visit the official Flutter website: https://flutter.dev/docs/get-started/install
2. Select your operating system (e.g., Windows).
3. Download the latest stable release of the Flutter SDK as a .zip archive.

Step 3: Extract Flutter SDK

1. Extract the downloaded ZIP file to a permanent location on your computer. Example location: C:\src\flutter
2. Avoid extracting it to system directories that require administrator permissions, such as C:\Program Files\.

Step 4: Set Environment Variable (Path)

1. Open **System Properties → Environment Variables**.
2. Under **System variables**, select the Path variable and click **Edit**.
3. Add the full path to the Flutter SDK's bin folder, for example:
4. C:\src\flutter\bin
5. Save and close all dialogs.

Step 5: Verify Flutter Installation

1. Open **Command Prompt** or **PowerShell**.

2. Run the following command to verify Flutter is correctly installed and see what else you need to set up:
3. flutter doctor
4. Review the output. The tool will identify missing dependencies or required tools such as Android Studio or device drivers.

Step 6: Install Required Dependencies

- Install **Android Studio** to get the Android SDK and emulator.
- Optionally, install **Visual Studio Code** for a lightweight development environment.
- Install **Google Chrome** if you intend to develop Flutter web applications.

Step 7: Install Flutter Plugins in Your IDE

- **Visual Studio Code:**
  Open VS Code → Extensions (Ctrl+Shift+X) → Search for and install the **Flutter** extension (which also installs the **Dart** extension).
- **Android Studio:**
  Open Android Studio → Preferences/Settings → Plugins → Search for **Flutter** → Install the plugin. The Dart plugin will be installed automatically as a dependency.

Step 8: Set Up Android Emulator (Optional)

1. Open Android Studio and launch the **AVD Manager** (Android Virtual Device Manager).
2. Create a new virtual device (e.g., Pixel 5 with API 33).
3. Start the emulator and ensure it runs properly.

Step 9: Run Your First Flutter Application

1. Open a terminal and create a new Flutter project:
2. flutter create my_app
3. Navigate into your project directory:
4. cd my_app
5. Run your Flutter app on the connected device or emulator:
6. flutter run

Test Installation

To confirm that your Flutter setup is fully functional, run:

flutter doctor

Make sure that all sections have a green checkmark, indicating your environment is correctly configured.

b) Write a simple Dart program to understand the language basics.

Program:

```
void main() {
  String name = 'John';
  int age = 20;

  print('Name: $name, Age: $age');

  if (age >= 18) {
    print('$name is an adult.');
  } else {
    print('$name is a minor.');
  }

  for (int i = 1; i <= 3; i++) {
    print('Count: $i');
  }

  greet(name);
}

void greet(String name) {
  print('Hello, $name!');
}
```

Output:

Name::John,Age:20

John is an adult.

Count:1

Count:2

Count:3

Hello,John!

Explanation of Components:

- **void main()**
  The entry point of every Dart program. The code inside main() runs when the program starts.
- **Variables (String name, int age)**
  Store data with specific types:
  - String holds text.
  - int holds whole numbers.

- **print()**
  Displays output in the console. Here it shows the name and age, and messages inside the condition and loop.
- **Conditional statement (if-else)**
  Executes different code based on a condition.
  If age is 18 or older, it prints that the person is an adult; otherwise, it prints that they are a minor.
- **for loop**
  Repeats a block of code multiple times.
  Here, it counts from 1 to 3, printing each number.
- **Function (greet)**
  A reusable block of code that takes a parameter (name) and prints a greeting message.
  It is called from main() with greet(name).

2. a) Explore various Flutter widgets (Text, Image, Container, etc.).

Aim: To write a dart program using flutter in Exploring various Flutter widgets (Text, Image, Container, etc.).

Description :

Container & Padding Widgets

1. The Container Widget

- **Convenience Wrapper:** A Container is a versatile layout widget used to wrap other widgets. It combines styling, sizing, and positioning properties into a single box model.
- **Sizing & Boundary Behavior:**
  - Without a Child: If a Container has no child widget assigned, it automatically expands to fill all available space offered by its parent (e.g., the full screen).
  - With a Child: When a child widget is added, the Container automatically shrinks and tightly wraps to match the dimensions of its child widget (plus any padding).

2. Spacing Concepts: Padding vs. Margin

- Padding: The internal space *inside* the container that surrounds its child content.
- Margin: The external space *outside* the container that separates it from adjacent widgets or screen edges.

3. Controlling Spacing with EdgeInsets

Spacing values for both padding and margin properties require the EdgeInsets class rather than plain numeric values:

- EdgeInsets.all(double value): Applies uniform spacing equal on all four sides (top, bottom, left, right).
- EdgeInsets.symmetric({horizontal, vertical}): Applies equal spacing symmetrically—horizontal sets left/right padding, and vertical sets top/bottom padding.
- EdgeInsets.fromLTRB(left, top, right, bottom): Allows individual control over every specific edge.

4. The Dedicated Padding Widget

- Purpose: A lightweight alternative used solely to add internal padding space around a child widget.
- When to Use Padding: Ideal when you only need to add spacing around a widget without extra styling properties like background color, borders, or margins.
- Limitations of Padding:
  - Does not support the margin property.
  - Does not support background color or decoration properties.

Container:

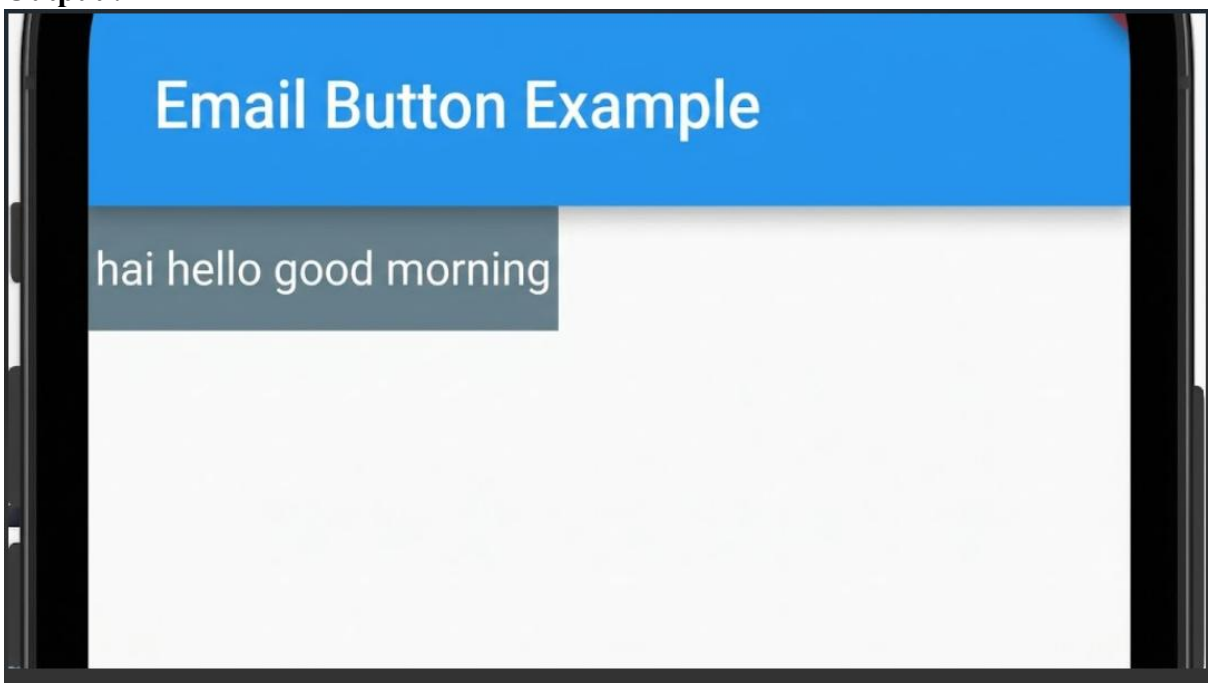
```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Email Button Example'),
        ),
        body: Container(
          color: Colors.grey[400],
          child: const Text('hai hello good morning'),
        ),
      ), // Fixed the extra closing parenthesis here
    );
  }
}
```

Output :



Other way :

```
body: Container(
          padding:EdgeInsets.fromLTRB(10.0, 20.0, 30.0,400),
          //padding: EdgeInsets.symmetric(horizontal: 30.0,vertical: 10.0),
          //padding: EdgeInsets.all(20.0),
          color: Colors.grey[400],
          child: const Text('hai hello good morning'),
        ),
```

With all padding and marigin :

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Email Button Example'),
        ),
        body: Container(
          padding:EdgeInsets.fromLTRB(10.0, 20.0, 30.0,400),
          margin: const EdgeInsets.all(20.0),
          //padding: EdgeInsets.symmetric(horizontal: 30.0,vertical: 10.0),
          //padding: EdgeInsets.all(20.0),
          color: Colors.grey[400],
          child: const Text('hai hello good morning'),
        ),
      ), // Fixed the extra closing parenthesis here
    );
  }
}
```

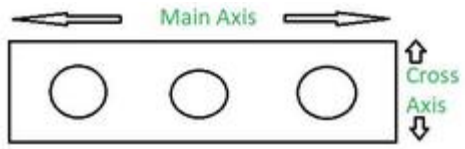
b) Implement different layout structures using Row, Column, and Stack widgets.

Aim: write a dart program using flutter lab to Implement different layout structures using Row, Column, and Stack widgets.

Row Layout Widget

1. Multi-Child Layout Framework

- **Single vs. Multi-Child:** Standard display widgets (e.g., Text, Image, Container) take a single child via the child property. Layout widgets like Row arrange multiple widgets using the children property.
- **Widget Lists:** The children property accepts a list of widgets (List<Widget>), allowing you to pass multiple UI elements separated by commas.
- **Horizontal Ordering:** A Row lays out its children linearly in a horizontal line from left to right.



2. Layout Axes Concept

Layout positioning in Flutter relies on two directional axes relative to the container type:

- **Main Axis:** The primary direction along which the layout flows.
    - For a Row, the **Main Axis is horizontal** (left-to-right).
- **Cross Axis:** The axis perpendicular to the main axis.
    - For a Row, the **Cross Axis is vertical** (top-to-bottom).

3. Main Axis Alignment (mainAxisAlignment**)**

Controls how extra horizontal space is distributed among and around child widgets along the main axis.

- MainAxisAlignment.start *(Default)*: Packs all children closely together at the beginning (left) of the row.
- MainAxisAlignment.end: Packs all children closely together at the end (right) of the row.
- MainAxisAlignment.center: Positions all children together in the middle of the row.
- MainAxisAlignment.spaceBetween: Distributes free space evenly between the children; the first child touches the start edge and the last child touches the end edge.
- MainAxisAlignment.spaceEvenly: Distributes free space evenly between all children as well as before the first and after the last child.
- MainAxisAlignment.spaceAround: Places equal space around every child (the space between adjacent children is double the space at the outer edges).

4. Cross Axis Alignment (crossAxisAlignment**)**

Controls how children are positioned or stretched vertically along the cross axis relative to the tallest item in the row.

- CrossAxisAlignment.center *(Default)*: Centers children vertically relative to the row's height.
- CrossAxisAlignment.start: Aligns children at the top edge of the row.
- CrossAxisAlignment.end: Aligns children at the bottom edge defined by the tallest child in the row.
- CrossAxisAlignment.stretch: Forces all child widgets to stretch and expand to fill the full vertical height available to the row.

Column Layout Widget

1. Multi-Child Vertical Framework

- **Vertical Stacking:** A Column widget lays out its list of children vertically in a single column from top to bottom.
- **Children Property (**children: <Widget>[]**):** Like Row, Column is a multi-child layout widget that takes a list of widgets passed to its children property.

2. Inverted Layout Axes

The directional axes for a Column are inverted relative to a Row:

- **Main Axis:** The primary direction of layout flow.
    - For a Column, the **Main Axis is vertical** (top-to-bottom).
- **Cross Axis:** The axis perpendicular to the main axis.
    - For a Column, the **Cross Axis is horizontal** (left-to-right).

3. Main Axis Alignment (mainAxisAlignment**)**

Controls how extra vertical space is distributed among and around child widgets along the vertical axis.

- MainAxisAlignment.start *(Default)*: Packs all children together at the top of the column.
- MainAxisAlignment.center: Centers all children vertically within the column.
- MainAxisAlignment.end: Packs all children together at the bottom of the column.
- MainAxisAlignment.spaceEvenly: Distributes free vertical space evenly between children, as well as above the top child and below the bottom child.
- MainAxisAlignment.spaceBetween: Distributes free vertical space evenly between children; the first child touches the top edge and the last child touches the bottom edge.
- MainAxisAlignment.spaceAround: Places equal vertical space above and below each child.

4. Cross Axis Alignment (crossAxisAlignment**)**

Controls how children are positioned or stretched horizontally relative to the widest child in the column (or relative to the available width of the column).

- CrossAxisAlignment.center *(Default)*: Centers children horizontally within the width of the column.
- CrossAxisAlignment.start: Aligns children to the left edge of the column.
- CrossAxisAlignment.end: Aligns children to the right edge of the column.
- CrossAxisAlignment.stretch: Forces all child widgets to stretch and expand across the full horizontal width available to the parent container.

5. Nesting Layouts (Combining Row **and** Column**)**

- **Layout Composition:** Complex UI layouts are built by nesting Row widgets inside Column widgets, or vice versa.
- **Width Behavior in Nesting:** When a Row is placed inside a Column, the Row naturally stretches across the horizontal width of the screen, expanding the cross axis boundaries of the parent Column.

Stack **Layout Widget**

1. Depth-Based Multi-Child Layout

- **Z-Axis Stacking:** Unlike Row (horizontal) and Column (vertical), the Stack widget arranges its children along the **Z-axis** (depth layer). Children are stacked on top of each other.
- **Layer Ordering (Back to Front):**
    - The **first child** in the children list is painted at the very bottom layer.
    - Subsequent children are rendered over previous layers.
    - The **last child** in the list is painted on the top layer.

2. Types of Stack Children

Children inside a Stack fall into two main categories:

A. Non-Positioned Children

- Widgets that are **not** wrapped in a Positioned or Align widget.
- By default, non-positioned children align themselves to the **top-left corner** of the Stack container unless modified by Stack.alignment.

B. Positioned Children

- Widgets wrapped inside a Positioned widget.
- Allows explicit user-defined positioning relative to the edges of the Stack.
- **Positioning Properties:**
    - **Vertical Axes:** top, bottom, height
    - **Horizontal Axes:** left, right, width

3. Core Stack **Properties**

- alignment: Controls how non-positioned or partially positioned children align within the stack (e.g., Alignment.center, Alignment.bottomLeft).
- fit: Controls how non-positioned children are sized relative to the stack bounds:
    - StackFit.loose *(Default)*: Children retain their own size constraints.
    - StackFit.expand: Forces children to expand and fill the maximum area available to the stack.

    - StackFit.passthrough: Passes the parent constraints directly to the stack's children.
- clipBehavior: Controls how children positioned outside the boundary of the stack are rendered:
    - Clip.hardEdge / Clip.antiAlias: Clips (hides) content that overflows outside the stack bounds.
    - Clip.none: Allows overflowing elements to remain visible outside the stack frame (replaces the deprecated overflow property in modern Flutter versions).
- textDirection: Controls the horizontal layout direction (e.g., TextDirection.ltr for left-to-right, TextDirection.rtl for right-to-left) when calculating edge alignments.

Programs :

Row padding :

```
import 'package:flutter/material.dart';

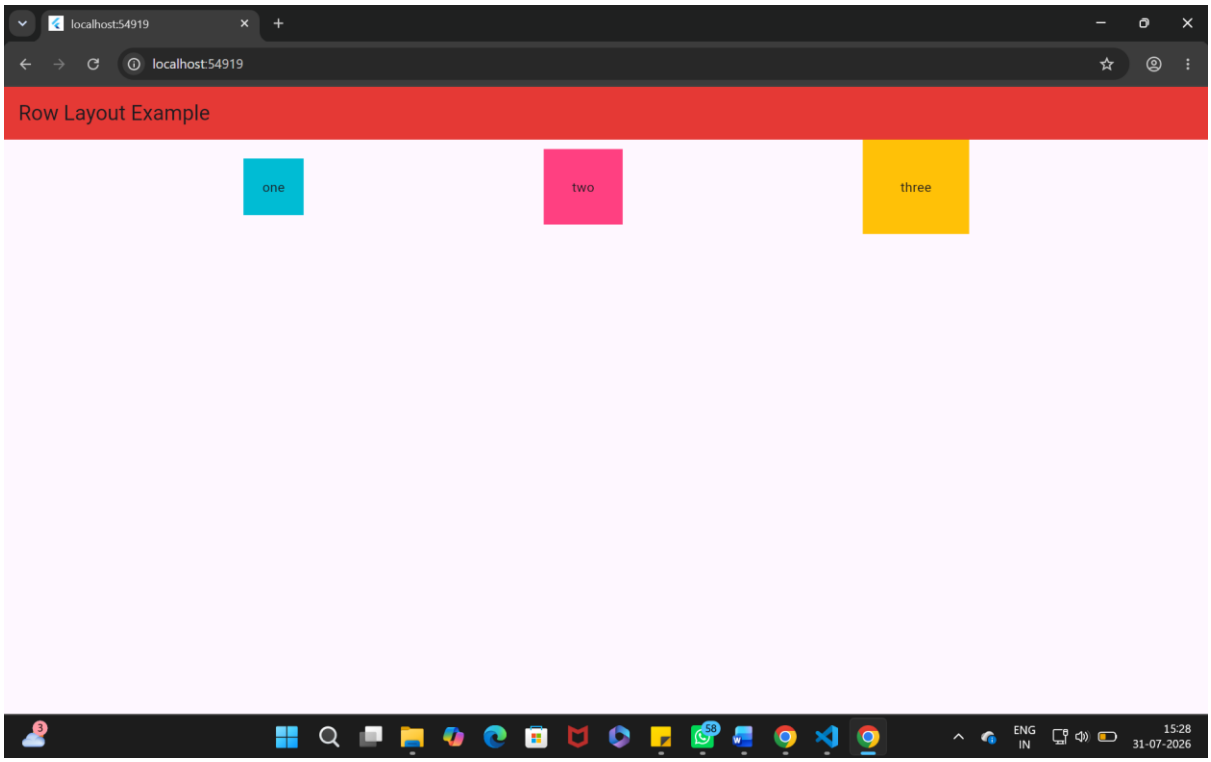
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Row Layout Example'),
          backgroundColor: Colors.red[600],
        ),
        body: Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: <Widget>[
            Container(
              padding: const EdgeInsets.all(20.0),
              color: Colors.cyan,
              child: const Text('one'),
            ), // Container
            Container(
              padding: const EdgeInsets.all(30.0),
```

```
            color: Colors.pinkAccent,
            child: const Text('two'),
          ), // Container
          Container(
            padding: const EdgeInsets.all(40.0),
            color: Colors.amber,
            child: const Text('three'),
          ), // Container
        ], // <Widget>[]
      ), // Row
    ), // Scaffold
  );
 }
}
```

Output:



Columns:

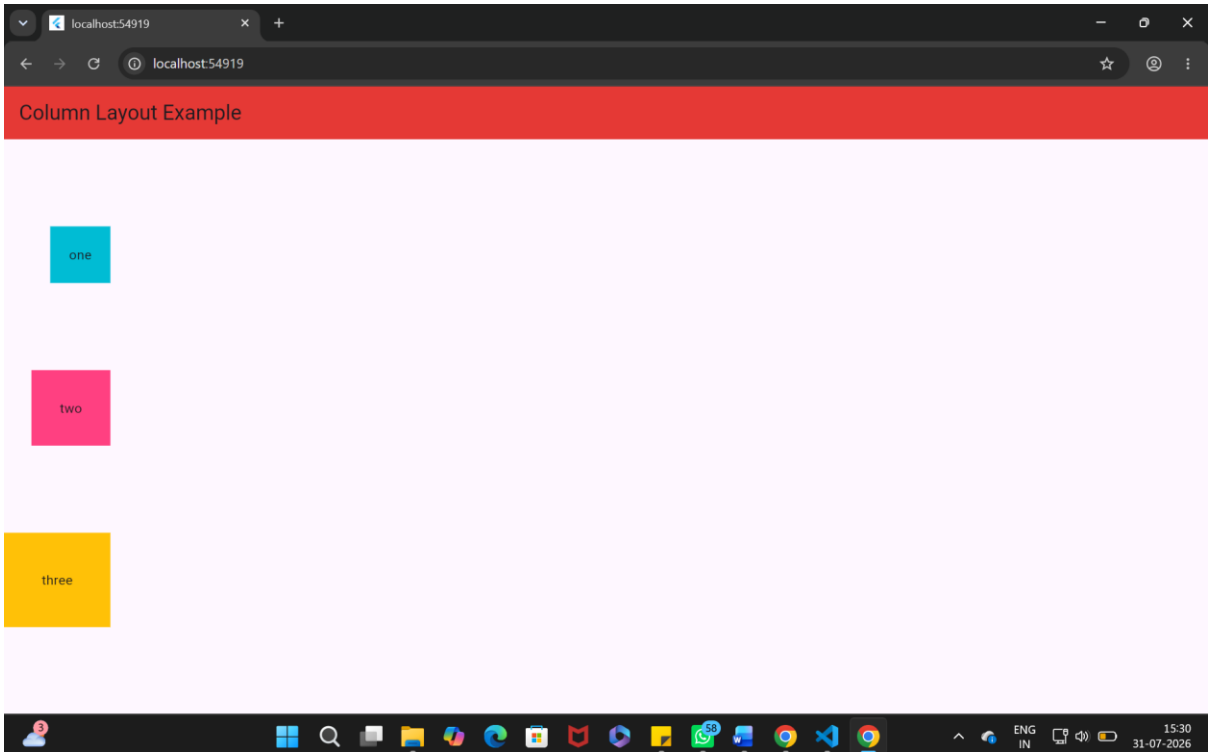
```
import 'package:flutter/material.dart';

void main() {
 runApp(const MyApp());
}

class MyApp extends StatelessWidget {
 const MyApp({super.key});
```

```
 @override
 Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Column Layout Example'),
        backgroundColor: Colors.red[600],
      ),
      body: Column(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        crossAxisAlignment: CrossAxisAlignment.end,
        children: <Widget>[
          Container(
            padding: const EdgeInsets.all(20.0),
            color: Colors.cyan,
            child: const Text('one'),
          ), // Container
          Container(
            padding: const EdgeInsets.all(30.0),
            color: Colors.pinkAccent,
            child: const Text('two'),
          ), // Container
          Container(
            padding: const EdgeInsets.all(40.0),
            color: Colors.amber,
            child: const Text('three'),
          ), // Container
        ], // <Widget>[]
      ), // Column
    ), // Scaffold
  );
 }
}
```

Output:



Stacked widget :

```
import 'package:flutter/material.dart';

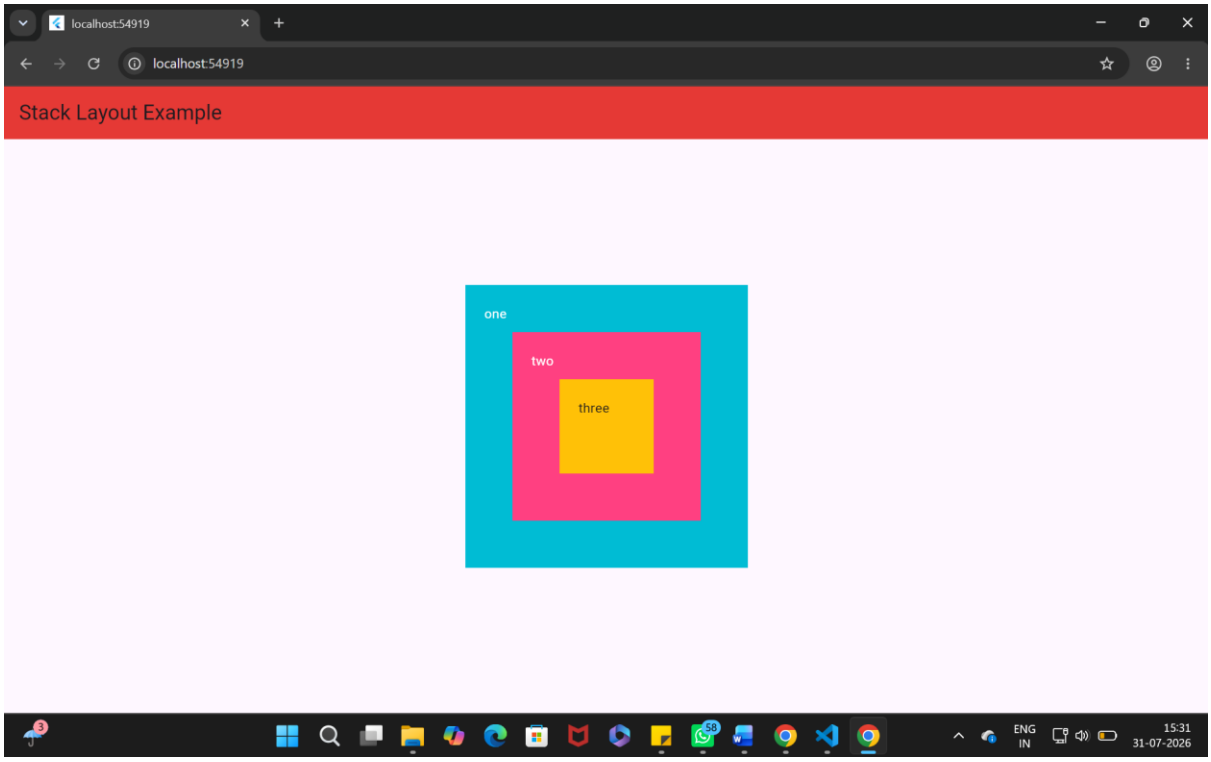
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Stack Layout Example'),
          backgroundColor: Colors.red[600],
        ),
        body: Center(
```

```
          child: Stack(
            alignment: Alignment.center,
            children: <Widget>[
              // The bottom layer (largest container)
              Container(
                width: 300,
                height: 300,
                color: Colors.cyan,
                padding: const EdgeInsets.all(20.0),
                child: const Text('one', style: TextStyle(color: Colors.white)),
              ),
              // The middle layer (medium container)
              Container(
                width: 200,
                height: 200,
                color: Colors.pinkAccent,
                padding: const EdgeInsets.all(20.0),
                child: const Text('two', style: TextStyle(color: Colors.white)),
              ),
              // The top layer (smallest container)
              Container(
                width: 100,
                height: 100,
                color: Colors.amber,
                padding: const EdgeInsets.all(20.0),
                child: const Text('three'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Output:



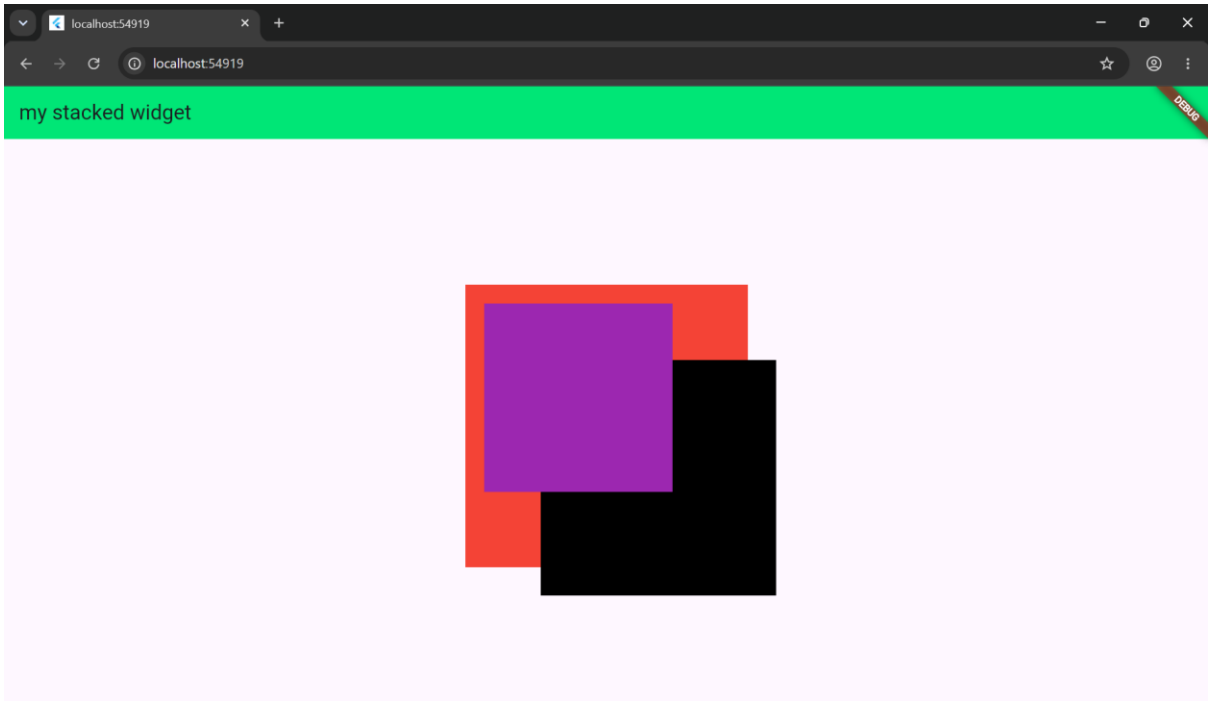
Stacked widget with positioned:

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('my stacked widget'),
          backgroundColor: Colors.greenAccent[400],
        ),
        body: Center(
          child: SizedBox(
            width: 300,
            height: 300,
            child: Center(
              child: Stack(
                fit: StackFit.expand,
                clipBehavior: Clip.none, // Replaces 'overflow: Overflow.visible'
                children: <Widget>[
                  Container(
```

```
                width: 300,
                height: 300,
                color: Colors.red,
              ),
              Positioned(
                top: 80,
                left: 80,
                child: Container(
                  width: 250,
                  height: 250,
                  color: Colors.black,
                ),
              ),
              Positioned(
                left: 20,
                top: 20,
                child: Container(
                  height: 200,
                  width: 200,
                  color: Colors.purple,
                ),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
 );
}
```

Output:


localhost:54919
localhost:54919
my stacked widget
DEBUG

a) Design a responsive UI that adapts to different screen sizes.

Aim : To design a responsive UI that adapts to different screen sizes using flutter lab using dart programming.

Description : we can build responsive applications with Flutter.

The LayoutBuilder:

Builds a widget tree that can depend on the parent widget's size. This is useful if we want to change or hide something depending on the parent size.

```
LayoutBuilder(
      builder: (BuildContext context, BoxConstraints constraints) {

       // constraints provide us with maxWidth,maxHeight etc, using
       // which we can show different widgets accordingly
       if (constraints.maxWidth > 600) {

         // as the width is greater than 600px,
         // we'll show wide screen container
         return _buildWideScreenContainers();
        } else {
         return _buildPortraitContainer();
        }
      },
),
```

Program :

```
import 'package:flutter/material.dart';

import 'package:flutter/material.dart';

void main() {
 runApp(const ResponsiveBodyApp());
}

class ResponsiveBodyApp extends StatelessWidget {
 const ResponsiveBodyApp({super.key});

 @override
 Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'Responsive Body Demo',
    theme: ThemeData(
     useMaterial3: true,
     colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
    ),
```

```
      home: const ResponsivePage(),
    );
  }
}

class ResponsivePage extends StatelessWidget {
  const ResponsivePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Adaptive Body Engine'),
        centerTitle: true,
        backgroundColor: Theme.of(context).colorScheme.primaryContainer,
      ),

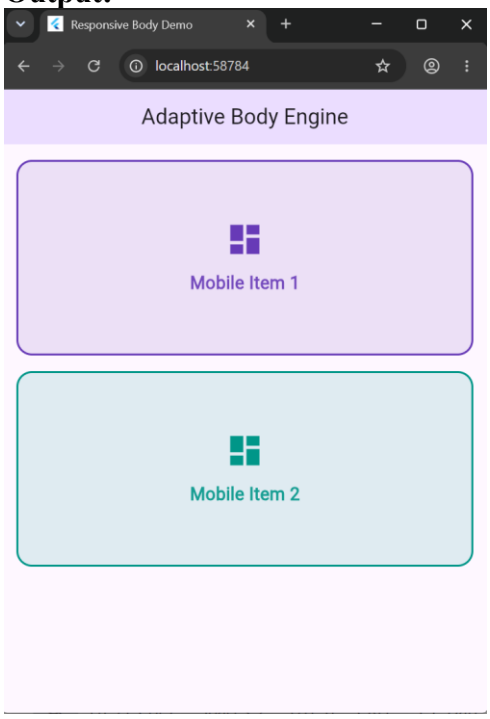
      body: LayoutBuilder(
        builder: (context, constraints) {
          // Breakpoint Condition: Narrow Screen (Mobile View)
          if (constraints.maxWidth < 600) {
            return ListView(
              padding: const EdgeInsets.all(16.0),
              children: [
                _buildCard('Mobile Item 1', Colors.deepPurple),
                const SizedBox(height: 16),
                _buildCard('Mobile Item 2', Colors.teal),
              ],
            );
          }

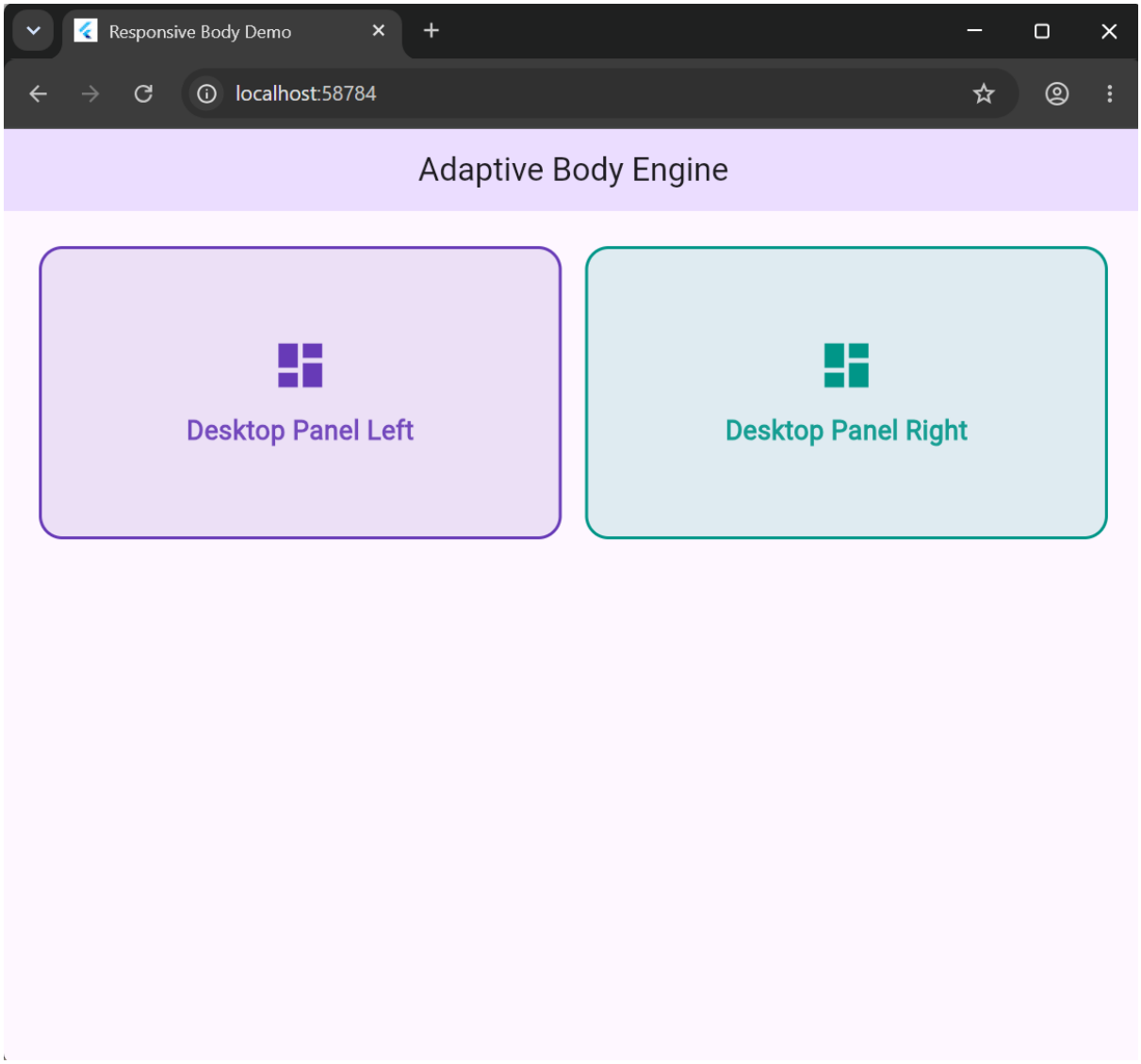
          // Breakpoint Condition: Wide Screen (Tablet / Desktop View)
          else {
            return Padding(
              padding: const EdgeInsets.all(24.0),
              child: Row(
                children: [
                  Expanded(child: _buildCard('Desktop Panel Left', Colors.deepPurple)),
                  const SizedBox(width: 16),
                  Expanded(child: _buildCard('Desktop Panel Right', Colors.teal)),
                ],
              ),
            );
          }
        },
      ),
    );
  }

  Widget _buildCard(String title, Color accentColor) {
    return Container(
      height: 200,
      decoration: BoxDecoration(
```

```
        color: accentColor.withOpacity(0.12),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: accentColor, width: 2),
      ),
      child: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(Icons.dashboard, size: 40, color: accentColor),
            const SizedBox(height: 12),
            Text(
              title,
              style: TextStyle(
                fontSize: 18,
                fontWeight: FontWeight.bold,
                color: accentColor.withOpacity(0.9),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:





b) Implement media queries and breakpoints for responsiveness.

Aim: Implement media queries and breakpoints for responsiveness.

Description :

MediaQuery

MediaQuery also lets us have the constraints. MediaQuery will not take constraints from the parent widget though, but instead, take it from the whole layout. Let's see an example using MediaQuery(), where we hide the AppBar() depending on the screen width.

```
class MediaQueryExample extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final screenW = MediaQuery.of(context).size.width;
    print(screenW);

    return Scaffold(
```

```
      // when screen's width is less than 600px,
      // it shows a appbar, when it's over 600px,
      // it hides it.
      appBar: screenW <= 600
          ? AppBar(
              title: Text("hai good morning"), backgroundColor: Colors.green)
          : null,
      body: Center(
        child: Text("Mediaquery example"),
      ),
    );
  }

}
```

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MediaQueryResponsiveApp());

class MediaQueryResponsiveApp extends StatelessWidget {
  const MediaQueryResponsiveApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: ResponsivePage(),
    );
  }
}

class ResponsivePage extends StatelessWidget {
  const ResponsivePage({super.key});

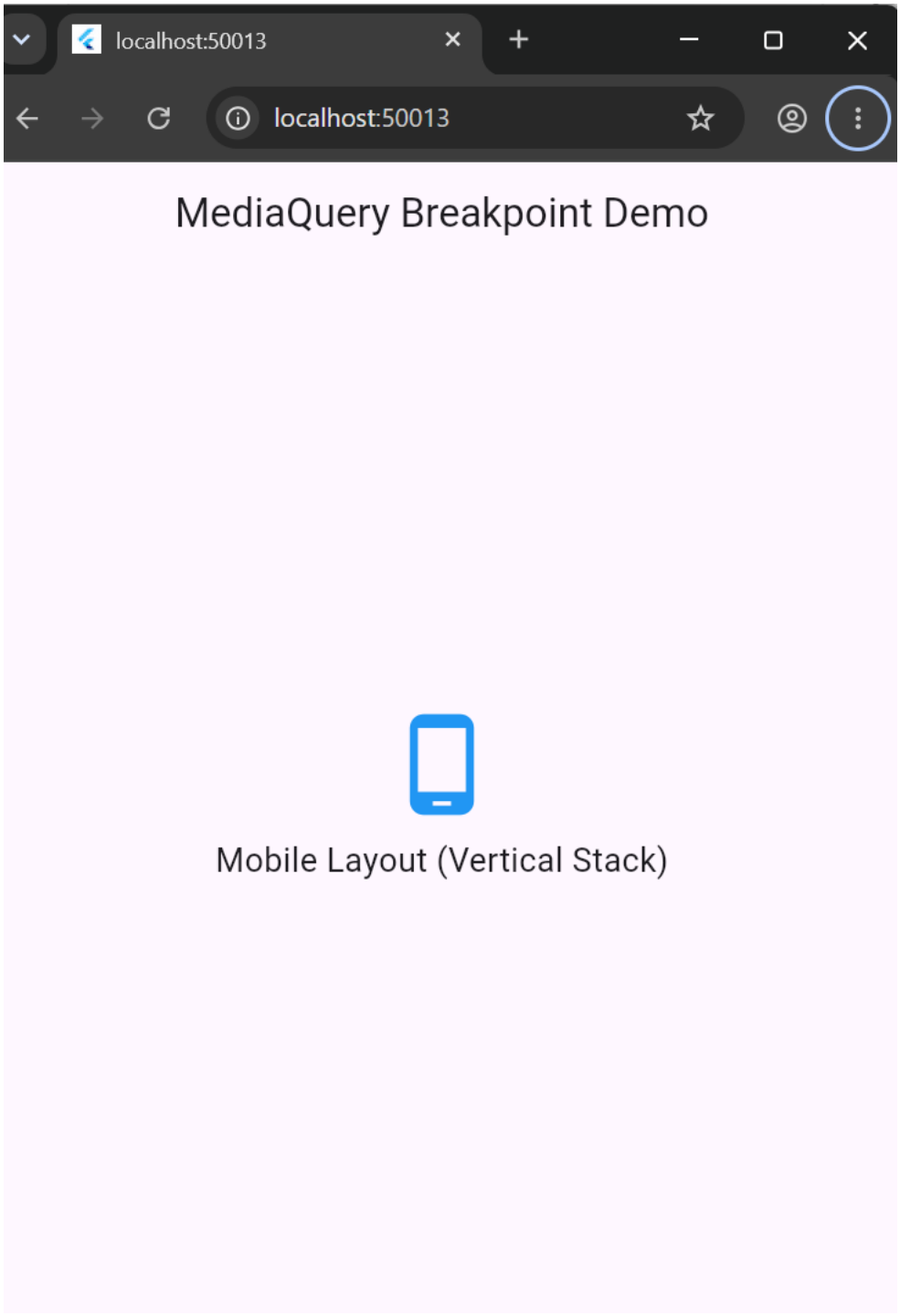
  @override
  Widget build(BuildContext context) {
    // 🌐 1. Fetch the screen width using MediaQuery
    double screenWidth = MediaQuery.of(context).size.width;

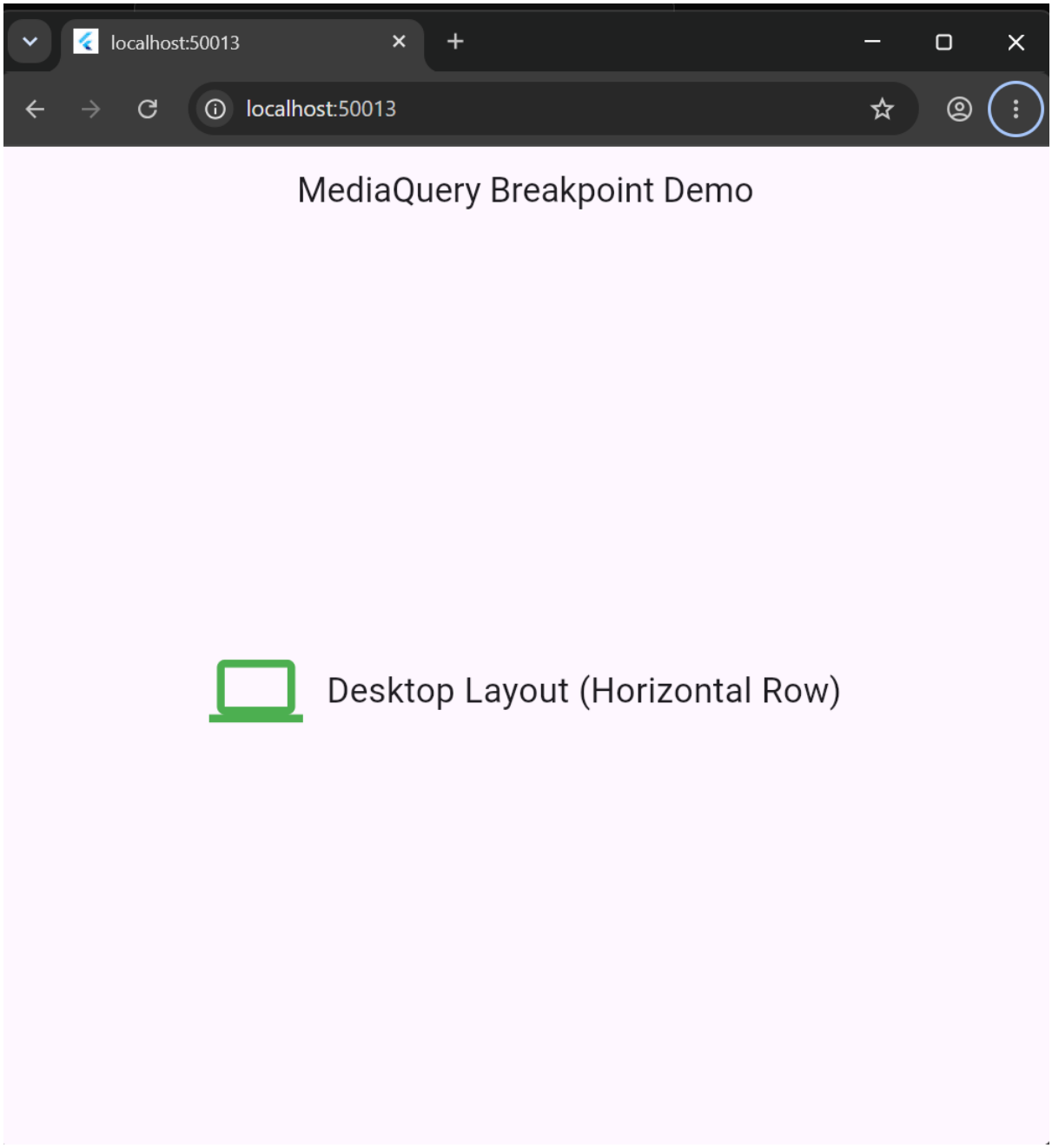
    return Scaffold(
      appBar: AppBar(
        title: const Text('MediaQuery Breakpoint Demo'),
        centerTitle: true,
      ),

      // ⚡ 2. The entire responsive shift happens right here inside the body property
      body: Center(
        child: screenWidth < 600
            ? const Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
```

```
                Icon(Icons.phone_android, size: 60, color: Colors.blue),
                SizedBox(height: 10),
                Text('Mobile Layout (Vertical Stack)', style: TextStyle(fontSize: 18)),
              ],
            )
          : const Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                Icon(Icons.computer, size: 60, color: Colors.green),
                SizedBox(width: 15),
                Text('Desktop Layout (Horizontal Row)', style: TextStyle(fontSize: 22)),
              ],
            ),
      ),
    );
  }
}
```

Output:


localhost:50013
localhost:50013
MediaQuery Breakpoint Demo
Mobile Layout (Vertical Stack)


localhost:50013
localhost:50013
MediaQuery Breakpoint Demo
Desktop Layout (Horizontal Row)

EXPERIMENT-4

1. a) Set up navigation between different screens using Navigator.

b) Implement navigation with named routes.

a) Set up navigation between different screens using Navigator.

Description:

In Flutter, navigation between different screens (or pages) is managed using the **Navigator** widget, which maintains a stack of routes representing the screens. By pushing a new route onto the stack, you move forward to a new screen, and by popping a route, you return to the previous screen. This stack-based navigation model allows for smooth transitions and back-button support. Developers define routes as widgets and use Navigator.push() to navigate to a new screen and Navigator.pop() to go back. This approach enables building multi-screen apps with clear flow and user control, supporting both simple navigation and complex routing scenarios such as passing data between screens and handling named routes.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const NavigationWithImagesApp());

class NavigationWithImagesApp extends StatelessWidget {
  const NavigationWithImagesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Navigator with Images Demo',
      home: FirstScreen(),
    );
  }
}

class FirstScreen extends StatelessWidget {
  const FirstScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Screen'),
      ),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
```

```
          Image.network(
                                                    'https://images.unsplash.com/photo-1506744038136-
46273834b3fb?auto=format&fit=crop&w=600&q=60',
            width: 300,
            height: 200,
            fit: BoxFit.cover,
          ),
          const SizedBox(height: 20),
          ElevatedButton(
            child: const Text('Go to Second Screen'),
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => const SecondScreen(),
                ),
              );
            },
          ),
        ],
      ),
    ),
  );
 }
}

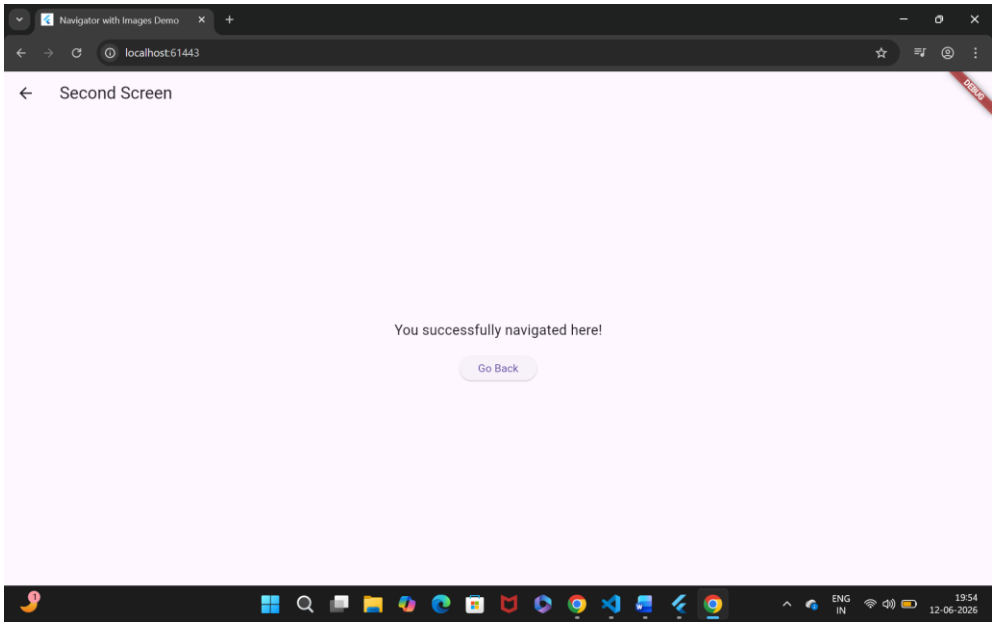
class SecondScreen extends StatelessWidget {
 const SecondScreen({super.key});

 @override
 Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
     title: const Text('Second Screen'),
    ),
    body: Center(
     child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
       const Text(
        'You successfully navigated here!',
        style: TextStyle(fontSize: 18),
       ),
       const SizedBox(height: 20),
       ElevatedButton(
        child: const Text('Go Back'),
        onPressed: () {
         Navigator.pop(context);
        },
       ),
      ],
```

```
        ),
      ),
    );
  }
}
```

Sample Output:





b) *Implement navigation with named routes.*

Description:

Flutter's **named routes** provide a clean and organized way to manage navigation in apps with multiple screens. Instead of directly pushing widget instances onto the navigation stack, named routes use string identifiers to reference screens. Developers define a map of route names to their corresponding widget builders in the MaterialApp (or CupertinoApp) configuration. Navigation is then performed using Navigator.pushNamed() and Navigator.pop(), which improves code readability and makes it easier to manage and scale complex navigation flows. Named routes also simplify deep linking and make it straightforward to pass arguments between screens. This method promotes better separation of concerns and centralizes route definitions for easier maintenance.

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const NavigationWithImagesApp());

class NavigationWithImagesApp extends StatelessWidget {
  const NavigationWithImagesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Named Routes Demo',
      initialRoute: '/',
      routes: {
        '/': (context) => const FirstScreen(),
        '/second': (context) => const SecondScreen(),
      },
    );
  }
}

class FirstScreen extends StatelessWidget {
  const FirstScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('First Screen')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Image.network(
              'https://images.unsplash.com/photo-1506744038136-
46273834b3fb?auto=format&fit=crop&w=600&q=60',
```

```
            width: 300,
            height: 200,
            fit: BoxFit.cover,
          ),
          const SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              // Navigating to '/second' and passing an argument string
              Navigator.pushNamed(
                context,
                '/second',
                arguments: 'Data sent from First Screen!',
              );
            },
            child: const Text('Go to Second Screen'),
          ),
        ],
      ),
    ),
  );
 }
}

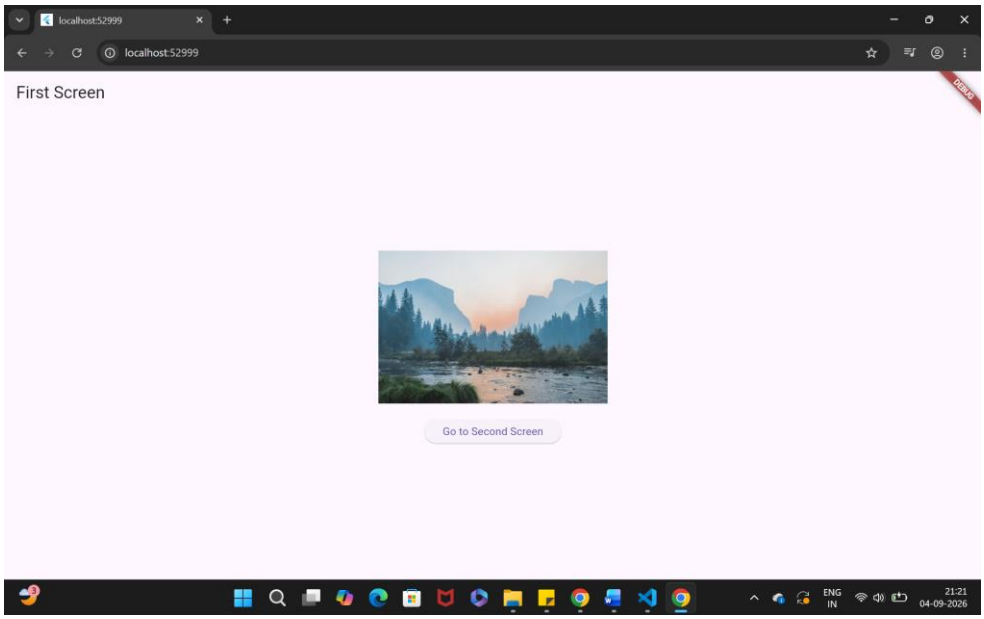
class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

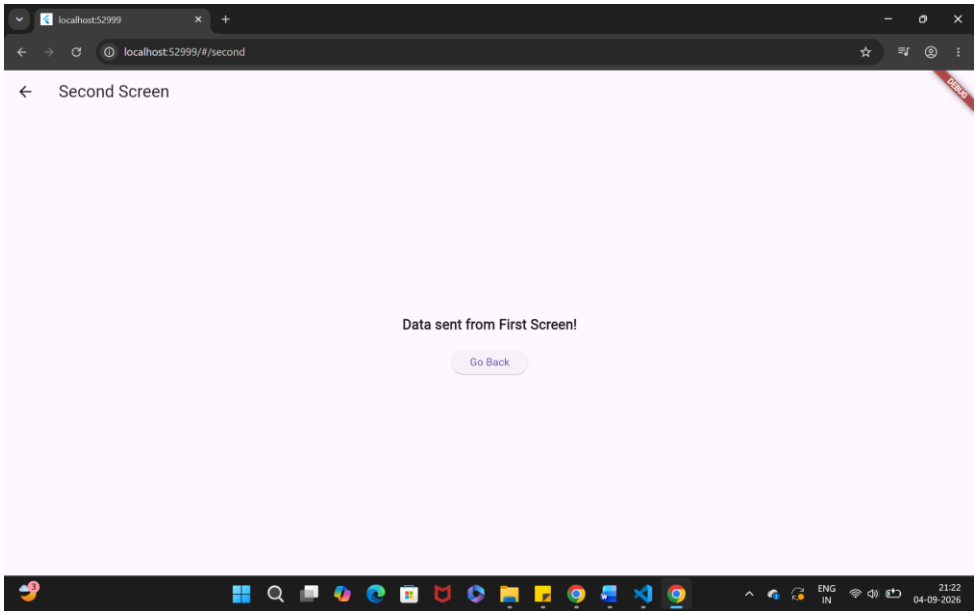
  @override
  Widget build(BuildContext context) {
    // Extracting the arguments passed via Navigator.pushNamed
    final String receivedData =
        ModalRoute.of(context)?.settings.arguments as String? ?? 'No Data Passed';

    return Scaffold(
      appBar: AppBar(title: const Text('Second Screen')),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(
              receivedData,
              style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Go Back'),
            ),
          ],
        ),
```

```
      ),
    );
  }
}
```

Sample Output:





Experiment-5

a) Learn about stateful and stateless widgets.
b) Implement state management using set State and Provider.

a) Learn about stateful and stateless widgets.

Description:

In Flutter, widgets are the fundamental building blocks of the user interface, and they are mainly classified into two types: Stateless widgets and Stateful widgets. A Stateless widget is immutable, meaning once it is built, it cannot change its state or data. These widgets are typically used to display static content such as text, icons, or images. Common examples include Text, Icon, and ElevatedButton (previously RaisedButton). On the other hand, a Stateful widget is mutable and can change its internal state during runtime using the setState() method. These widgets are suitable for handling dynamic content that needs to update or respond to user interactions, such as counters, forms, or animations. Examples include Checkbox, TextField, and Slider. Together, these two types of widgets allow Flutter developers to build both static and interactive parts of an application.

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const WidgetTypesApp());
}

class WidgetTypesApp extends StatelessWidget {
  const WidgetTypesApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Widget Types Demo',
      home: Scaffold(
        appBar: AppBar(title: const Text('Stateless vs Stateful')),
        body: const Center(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              MyStatelessWidget(),
              SizedBox(height: 20),
              MyStatefulWidget(),
            ],
          ),
        ),
      ),
    );
```

```
  }
}

// Immutable Widget - Contents cannot change after initial render
class MyStatelessWidget extends StatelessWidget {
  const MyStatelessWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.orange[200],
      margin: const EdgeInsets.all(12),
      child: const Padding(
        padding: EdgeInsets.all(16.0),
        child: Text(
          "I am Stateless 😎",
          style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
        ),
      ),
    );
  }
}

// Mutable Widget - Internal state can modify dynamically at runtime
class MyStatefulWidget extends StatefulWidget {
  const MyStatefulWidget({super.key});

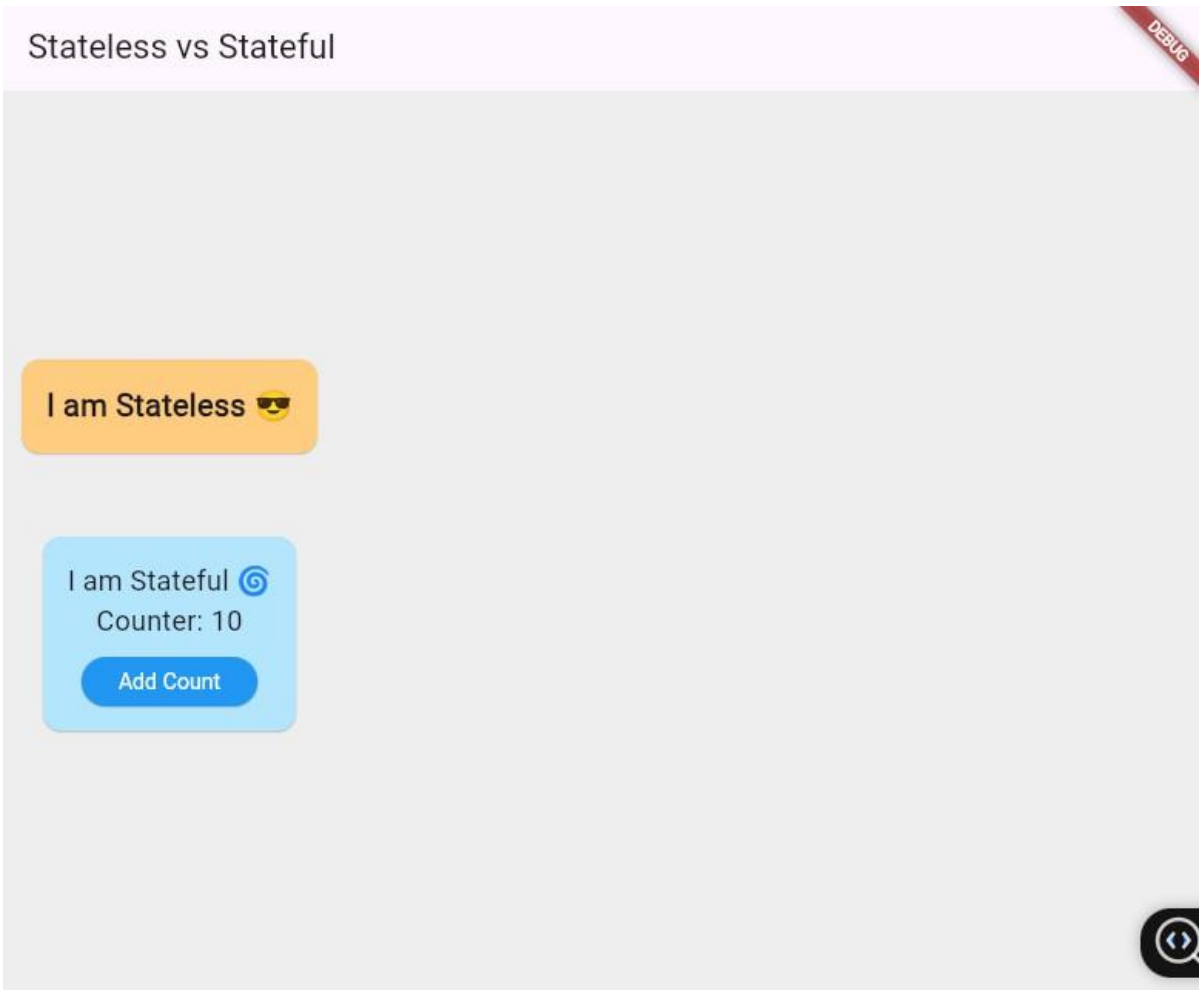
  @override
  State<MyStatefulWidget> createState() => _MyStatefulWidgetState();
}

class _MyStatefulWidgetState extends State<MyStatefulWidget> {
  int counter = 0;

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.lightBlue[100],
      margin: const EdgeInsets.all(12),
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            Text(
              "I am Stateful 🌀\nCounter: $counter",
              style: const TextStyle(fontSize: 18),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 10),
```

```
            ElevatedButton(
              style: ElevatedButton.styleFrom(
                backgroundColor: Colors.blue,
                foregroundColor: Colors.white,
              ),
              onPressed: () {
                setState(() {
                  counter++;
                });
              },
              child: const Text("Add Count"),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output:



b) *Implement state management using set State and Provider.*

Description :

In Flutter, **state management** is the technique used to control how data is stored and updated within an app. The two simplest ways to handle this are **setState** and **Provider**. Using **setState**, the state is managed locally inside a single widget, and UI updates happen only within that widget when setState() is called. This approach is best for small, isolated changes like counters, toggles, or form fields. On the other hand, **Provider** is used for managing and sharing state globally across the entire app. It stores the state in a separate class and makes it accessible to multiple widgets, ensuring consistent updates everywhere. This is useful for features like a shopping cart, authentication, or themes where many screens depend on the same data. Together, setState is suitable for **local state**, while Provider is ideal for **global state sharing** in real-world applications.

Program:

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => CartProvider(),
      child: const StateManagementApp(),
    ),
  );
}

class StateManagementApp extends StatelessWidget {
  const StateManagementApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'State Management Demo',
      home: DefaultTabController(
        length: 2,
        child: Scaffold(
          appBar: AppBar(
            title: const Text("Cart Demo"),
            bottom: const TabBar(
              tabs: [
                Tab(text: "setState (Local)"),
                Tab(text: "Provider (Global)"),
              ],
            ),
          ),
          body: const TabBarView(
```

```
            children: [
              LocalCart(),
              GlobalCart(),
            ],
          ),
        ),
      ),
    );
  }
}

// 1. Local State Scope using standard setState()
class LocalCart extends StatefulWidget {
  const LocalCart({super.key});

  @override
  State<LocalCart> createState() => _LocalCartState();
}

class _LocalCartState extends State<LocalCart> {
  int apples = 0;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text("Apples: $apples", style: const TextStyle(fontSize: 24)),
          const SizedBox(height: 10),
          ElevatedButton(
            onPressed: () {
              setState(() {
                apples++;
              });
            },
            child: const Text("Add Apple"),
          ),
        ],
      ),
    );
  }
}

// 2. State Controller for Global Data sharing across decoupled elements
class CartProvider extends ChangeNotifier {
  int items = 0;

  void add() {
    items++;
    notifyListeners(); // Requests subscribers to rebuild
```

```
  }
}

class GlobalCart extends StatelessWidget {
  const GlobalCart({super.key});

  @override
  Widget build(BuildContext context) {
    final cart = Provider.of<CartProvider>(context);

    return Center(
      child: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          Text("Cart Items: ${cart.items}", style: const TextStyle(fontSize: 24)),
          const SizedBox(height: 10),
          ElevatedButton(
            onPressed: cart.add,
            child: const Text("Add Global Item"),
          ),
        ],
      ),
    );
  }
}
```

To add the package to your project, you will need to open your project's directory, update the configuration file, and pull down the library from the official Dart repository (pub.dev).
Here is exactly how to do it step-by-step:

Step 1: Open pubspec.yaml

In your Flutter project's root folder, locate the file named **pubspec.yaml**. This file handles all your project's configurations and external libraries.

Step 2: Add the Dependency

Scroll down until you find the dependencies: section. Add provider: ^6.1.2 directly underneath it.

⚠️ **Crucial Note:** Yaml files are extremely strict about indentation. Make sure provider: is indented with exactly **two spaces**, matching the alignment of flutter:. Do not use tabs!

It should look like this:

YAML

```
name: my_responsive_app
description: "A new Flutter project."
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.2.0 <4.0.0'

dependencies:
  flutter:
```

sdk: flutter
provider: ^6.1.2

Step 3: Download the Package

Now you need to tell Flutter to download the new files. You can do this in one of two ways:

- **Via Terminal:** Open your terminal in your project's root folder and run:

Bash

flutter pub get

- **Via VS Code / Android Studio:** If you are using an IDE, simply saving the pubspec.yaml file (Ctrl+S or Cmd+S) will automatically trigger flutter pub get for you.

Step 4: Import it in your Code

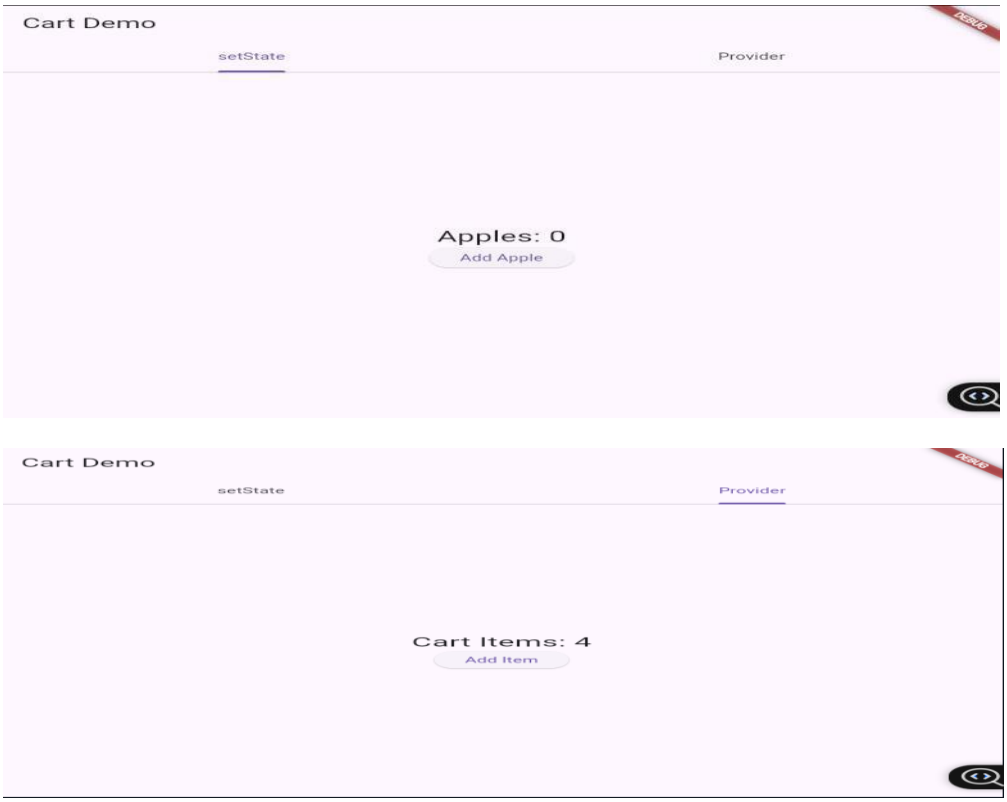
Once the download finishes, go to your Dart file where you pasted the Experiment 5(b) code. Make sure this import statement is at the very top of your file:

Dart

import 'package:provider/provider.dart';

If you had any red squiggly lines underneath ChangeNotifierProvider or Provider.of, they will now instantly disappear, and your app is ready to run!

Output :



Experiment – 6

a) Create custom widgets for specific UI elements.
b) Apply styling using themes and custom styles.

a) Create custom widgets for specific UI elements.

Description:

In Flutter, **custom widgets** allow developers to create reusable UI components that make the app more organized, maintainable, and scalable. Instead of repeating the same design multiple times, a custom widget can be defined once and reused across the app with different data or properties. For example, in a profile app, a ProfileCard widget can display a user's name, email, and avatar, while an ActionButton widget can represent reusable buttons like *Follow* or *Message*. This approach not only reduces code duplication but also makes the UI easier to modify and extend, which is especially useful in real-life apps like social media or e-commerce platforms.

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const CustomWidgetsApp());
}

class CustomWidgetsApp extends StatelessWidget {
  const CustomWidgetsApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Custom Widgets Demo',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Custom UI Elements'),
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
        body: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              // Reusing our custom card component with unique data
              const ProfileCard(
                name: "John Doe",
                email: "john.doe@example.com",
              ),
              const SizedBox(height: 20),
              // Reusing our custom button with independent properties
              ActionButton(
                text: "Follow",
                color: Colors.green,
                onPressed: () {
                  debugPrint("Follow button clicked");
                },
              ),
            ],
          ),
        ),
      ),
    );
  }
}

// 🗃️ CUSTOM REUSABLE WIDGET: ProfileCard
class ProfileCard extends StatelessWidget {
  final String name;
```

```
  final String email;

  // Modern syntax constructor utilizing super.key
  const ProfileCard({
    super.key,
    required this.name,
    required this.email,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 4,
      margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      child: ListTile(
        leading: const CircleAvatar(
          backgroundColor: Colors.blue,
          child: Icon(Icons.person, color: Colors.white),
        ),
        title: Text(
          name,
          style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
        ),
        subtitle: Text(email),
      ),
    );
  }
}

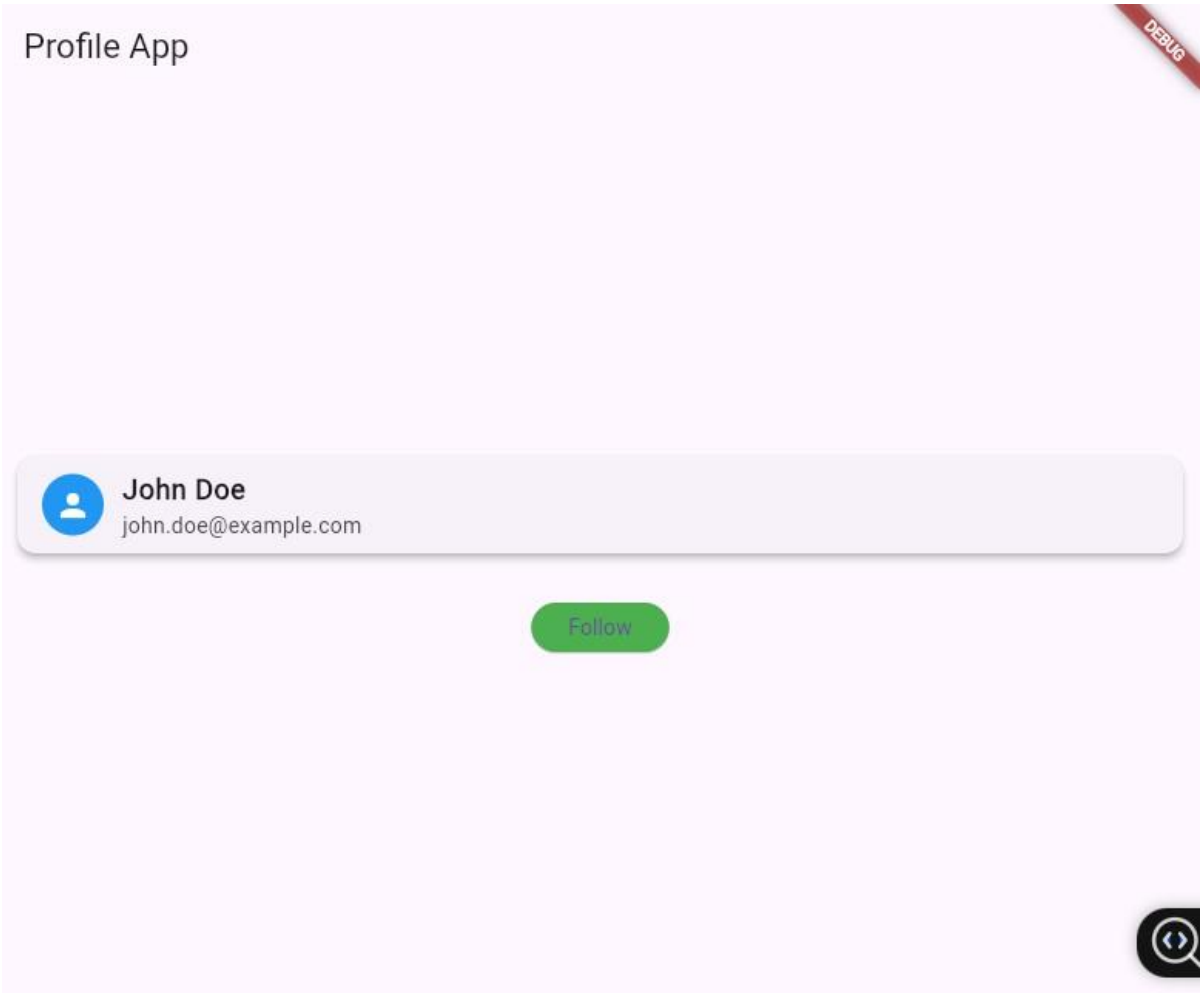
// 🎯 CUSTOM REUSABLE WIDGET: ActionButton
class ActionButton extends StatelessWidget {
  final String text;
  final Color color;
  final VoidCallback onPressed;

  const ActionButton({
    super.key,
    required this.text,
    required this.color,
    required this.onPressed,
  });

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      style: ElevatedButton.styleFrom(
        backgroundColor: color,
        foregroundColor: Colors.white, // Standard white text color
        padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(8),
```

```
        ),
      ),
      onPressed: onPressed,
      child: Text(
        text,
        style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ),
    );
  }
}
```

Output:



b) *Apply styling using themes and custom styles.*

Description :

In Flutter, **styling with themes and custom styles** helps maintain a consistent design across the entire app. Instead of manually styling each widget, developers can define a ThemeData that includes colors, fonts, and widget styles like buttons or text. This way, all widgets automatically follow the same design rules, making the UI look professional and uniform. For example, you can set a global button shape or text font once in the theme, and it will apply everywhere in the app. Custom styles can also be created for special widgets when needed, ensuring flexibility along with consistency. This approach reduces repetitive code, simplifies maintenance, and makes it easier to apply branding or switch between light and dark modes.

Program :

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp()); // Added const

class MyApp extends StatefulWidget {
  const MyApp({super.key}); // Added modern constructor

  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  bool isDark = false;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Theme Demo',
      // Light Mode Configuration
      theme: ThemeData(
        brightness: Brightness.light,
        primarySwatch: Colors.blue,
      ),
      // Dark Mode Configuration
      darkTheme: ThemeData(
        brightness: Brightness.dark,
        primarySwatch: Colors.deepPurple,
      ),
      // Active theme switcher state tracking
      themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
      // 💡 FIXED: Passing state variables down to a child widget with its own sub-context
      home: ThemeDemoPage(
        isDark: isDark,
        onThemeChanged: () {
          setState(() {
            isDark = !isDark;
```

```
          });
        },
      ),
    );
  }
}

// Moved the screen UI out into its own widget to ensure Theme.of(context) works perfectly
class ThemeDemoPage extends StatelessWidget {
  final bool isDark;
  final VoidCallback onThemeChanged;

  const ThemeDemoPage({
    super.key,
    required this.isDark,
    required this.onThemeChanged,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Theme Demo")),
      body: Center(
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            // 💡 This now looks up and successfully finds the MaterialApp theme rules!
            Text(
              "Hello Flutter!",
              style: Theme.of(context).textTheme.titleLarge,
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: onThemeChanged,
              child: Text(isDark ? "Switch to Light" : "Switch to Dark"),
            ),
          ],
        ),
      ),
    );
  }
}
```

Output :


Theme Demo
DEBUG
Hello Flutter!
Switch to Dark


Theme Demo
DEBUG
Hello Flutter!
Switch to Light

EXPERIMENT-7

a) *Design a form with various input fields.*

b) Implement form validation and error handling.

a) *Design a form with various input fields.*

Aim:

To design and implement a responsive mobile application form in Flutter that collects student information (Name, Roll Number, and Gender), performs client-side input validation, and displays the submitted data using an alert dialog.

Description / Working Principle:

- State Management: The application uses a StatefulWidget (StudentForm) to dynamically manage user inputs and update the UI state when selections change.
- Form & Validation: A Form widget coupled with a GlobalKey<FormState> is utilized to group input fields, handle validation logic, and trigger data saving collectively.
- Input Fields:
  - TextFormField is used for capturing text input (Name) and numeric input (Roll Number), complete with null-checks and empty-string error validations.
  - RadioListTile components handle the single-selection gender option (Male / Female).
- Action & Output: Clicking the Submit button invokes _formKey.currentState!.validate(). If all fields are valid, the form saves the data into local variables and triggers an AlertDialog to display the formatted summary of the entered student details.

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Student Form',
      home: StudentHomeScreen(),
    );
  }
}

class StudentHomeScreen extends StatelessWidget {
  const StudentHomeScreen({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Student Details Form')),
      body: const SafeArea(
        child: SingleChildScrollView(
          padding: EdgeInsets.all(16.0),
          child: StudentForm(),
        ),
      ),
    );
  }
}

class StudentForm extends StatefulWidget {
  const StudentForm({super.key});

  @override
  State<StudentForm> createState() => _StudentFormState();
}

class _StudentFormState extends State<StudentForm> {
  final _formKey = GlobalKey<FormState>();
  String _name = '';
  String _rollNo = '';
  String _gender = 'Male';

  @override
  Widget build(BuildContext context) {
    return Form(
      key: _formKey,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Name field
          TextFormField(
            decoration: const InputDecoration(labelText: 'Name'),
            onSaved: (value) => _name = value ?? '',
            validator: (value) =>
                value == null || value.trim().isEmpty ? 'Please enter your name' : null,
          ),
          const SizedBox(height: 16),

          // Roll No field
          TextFormField(
            decoration: const InputDecoration(labelText: 'Roll No'),
            keyboardType: TextInputType.number,
            onSaved: (value) => _rollNo = value ?? '',
            validator: (value) =>
                value == null || value.trim().isEmpty ? 'Please enter your roll number' : null,
          ),
          const SizedBox(height: 20),
```

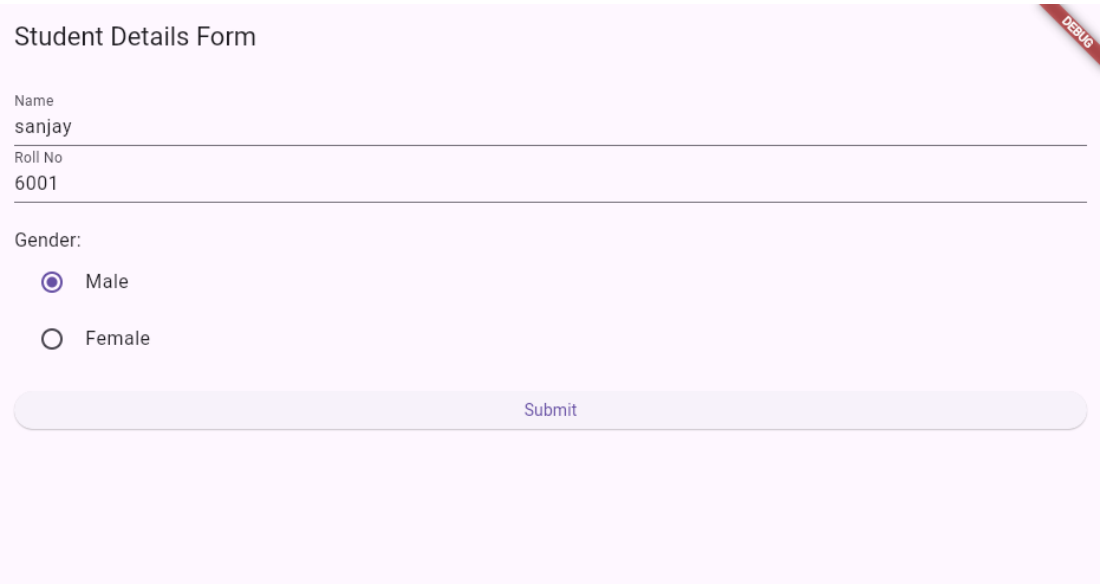
```
// Gender selection header
const Text('Gender:', style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),

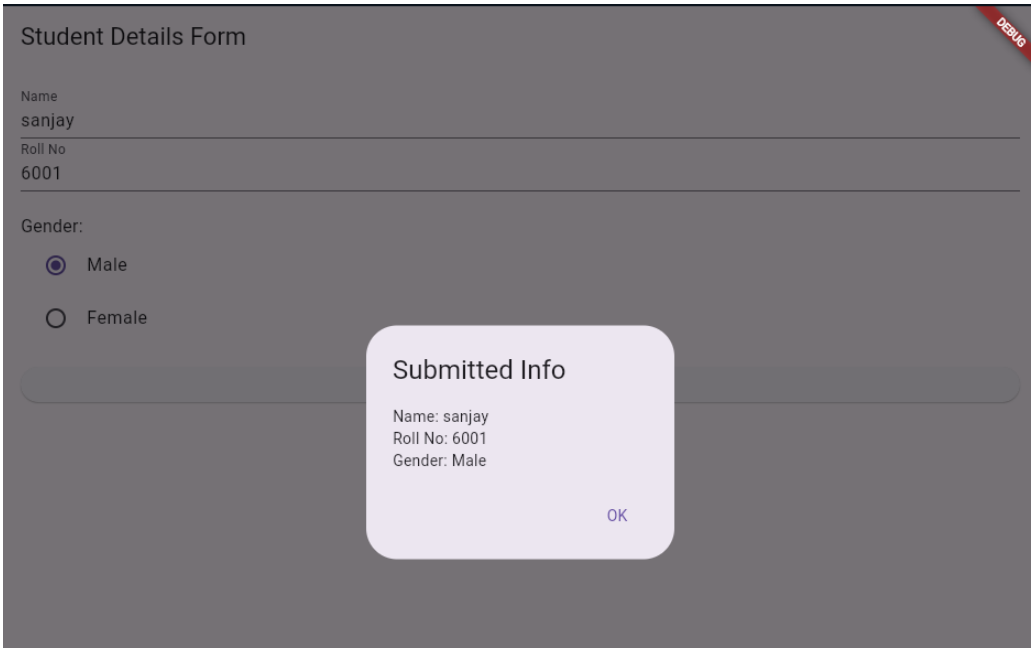
// Modern RadioGroup wrapper replacing individual groupValue/onChanged
RadioGroup<String>(
  groupValue: _gender,
  onChanged: (value) {
    if (value != null) {
      setState(() {
        _gender = value;
      });
    }
  },
  child: const Column(
    children: [
      RadioListTile<String>(
        title: Text('Male'),
        value: 'Male',
      ),
      RadioListTile<String>(
        title: Text('Female'),
        value: 'Female',
      ),
    ],
  ),
),
const SizedBox(height: 30),

// Submit button
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      _formKey.currentState!.save();
      showDialog(
        context: context,
        builder: (_) => AlertDialog(
          title: const Text('Submitted Info'),
          content: Text(
            'Name: $_name\nRoll No: $_rollNo\nGender: $_gender',
          ),
          actions: [
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('OK'),
            ),
          ],
        ),
      );
    }
  },
```

```
            child: const Text('Submit'),
          ),
        ],
      ),
    );
  }
}
```

Output:


Student Details Form
DEBUG
Name
sanjay
Roll No
6001
Gender:
Male
Female
Submit


Student Details Form
DEBUG
Name
sanjay
Roll No
6001
Gender:
Male
Female
Submitted Info
Name: sanjay
Roll No: 6001
Gender: Male
OK

b) *Implement form validation and error handling.*

Aim:

To design and develop a Flutter application featuring a form with multiple input fields, custom validator logic, text editing controllers, and real-time client-side error handling (including regular expression email verification).

Description / Working Principle:

- *State & Controllers: Uses TextEditingController instances (_nameController and _emailController) to track and extract input values from the text form fields, and properly disposes of them to prevent memory leaks.*
- *Form Key & Validation State: Utilizes a GlobalKey<FormState> to uniquely identify the Form widget, allowing the application to validate all child inputs simultaneously via _formKey.currentState!.validate().*
- *Custom Validators:*
    - *_validateName: Checks if the user-entered text is null or empty, returning an error message if invalid.*
    - *_validateEmail: Combines an emptiness check with a custom regular expression (RegExp(r'^[^@]+@[^@]+\.[^@]+')) to ensure the email address adheres to a standard structural format.*
- *Feedback Mechanism: When the Submit button is pressed, the form runs validation checks. If successful, a feedback notification (SnackBar) appears dynamically at the bottom of the screen confirming successful submission.*

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      debugShowCheckedModeBanner: false,
      title: 'Form Validation Demo',
      home: const FormValidationPage(),
    );
  }
}

class FormValidationPage extends StatefulWidget {
  const FormValidationPage({super.key});

  @override
  State<FormValidationPage> createState() => _FormValidationPageState();
}

class _FormValidationPageState extends State<FormValidationPage> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _emailController = TextEditingController();

  @override
  void dispose() {
    _nameController.dispose();
    _emailController.dispose();
    super.dispose();
  }

  String? _validateName(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Name is required';
    }
    return null;
  }

  String? _validateEmail(String? value) {
    if (value == null || value.trim().isEmpty) {
      return 'Email is required';
    }
    // Basic email regex
    final emailRegex = RegExp(r'^[^@]+@[^@]+\.[^@]+');
    if (!emailRegex.hasMatch(value)) {
      return 'Enter a valid email';
    }
    return null;
  }

  void _submitForm() {
    if (_formKey.currentState!.validate()) {
      // If the form is valid, show a snackbar
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Form submitted successfully!')),
      );
```

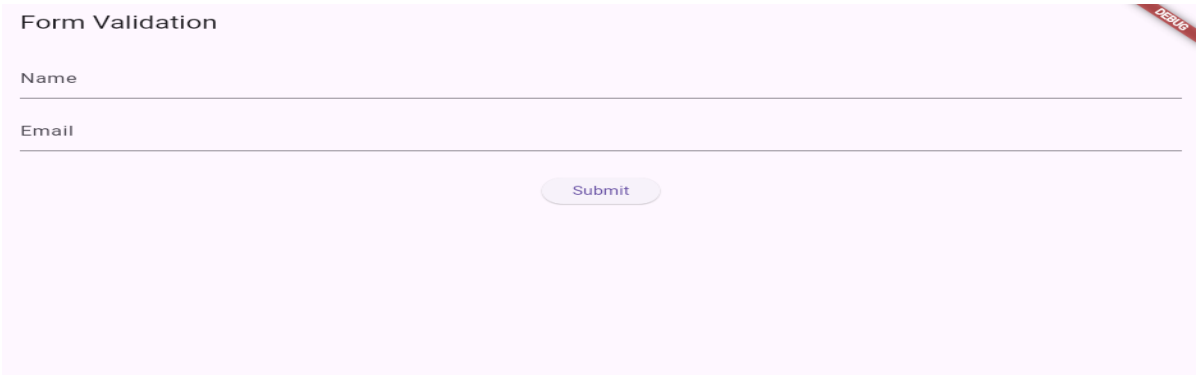
```
  }
 }

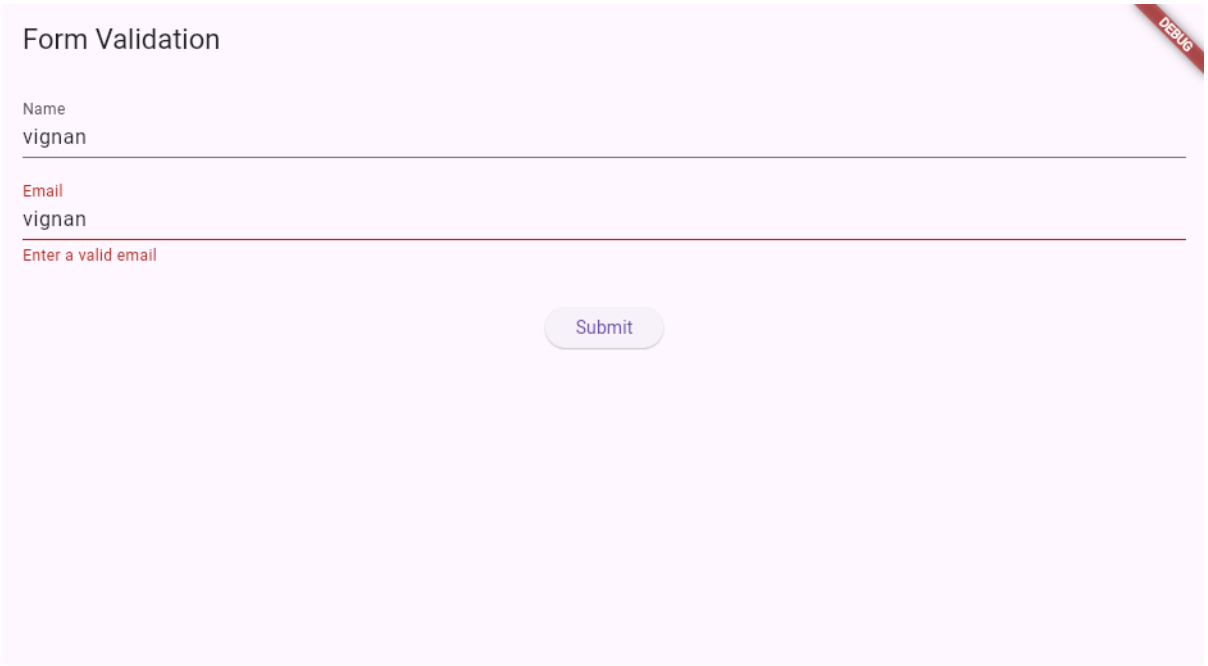
 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(title: const Text('Form Validation Demo')),
   body: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Form(
     key: _formKey,
     child: Column(
      children: [
       // Name Field
       TextFormField(
        controller: _nameController,
        decoration: const InputDecoration(labelText: 'Name'),
        validator: _validateName,
       ),
       const SizedBox(height: 16),

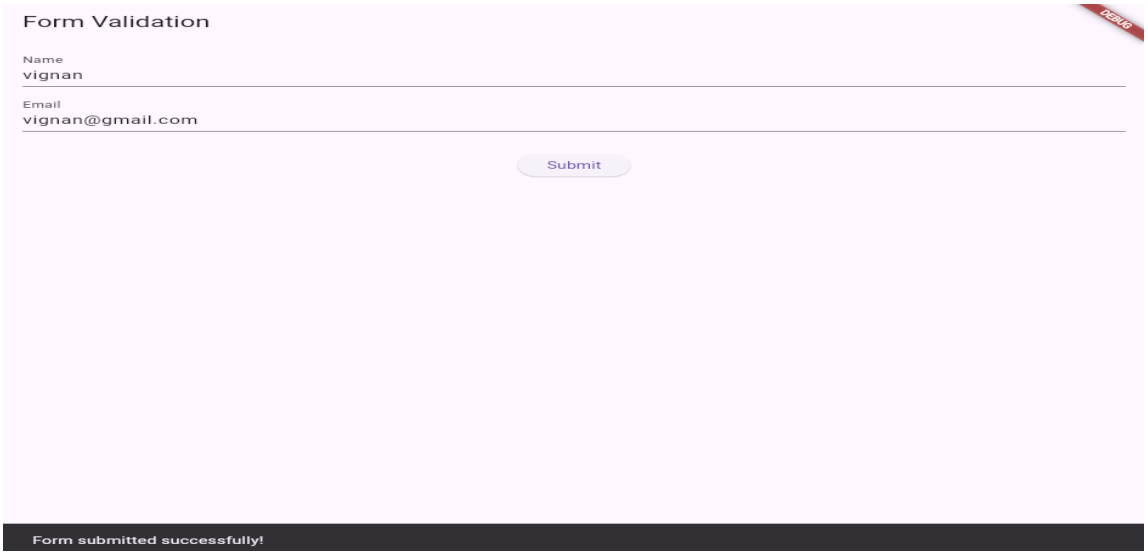
       // Email Field
       TextFormField(
        controller: _emailController,
        decoration: const InputDecoration(labelText: 'Email'),
        keyboardType: TextInputType.emailAddress,
        validator: _validateEmail,
       ),
       const SizedBox(height: 32),

       // Submit Button
       ElevatedButton(
        onPressed: _submitForm,
        child: const Text('Submit'),
       ),
      ],
     ),
    ),
   ),
  );
 }
}
```

Output:


Form Validation
DEBUG
Name
Email
Submit


Form Validation
DEBUG
Name
vignan
Email
vignan
Enter a valid email
Submit


Form Validation
DEBUG
Name
vignan
Email
vignan@gmail.com
Submit
Form submitted successfully!

EXPERIMENT-8

a) Add animations to UI elements using Flutter's animation framework.
b) Experiment with different types of animations (fade, slide, etc.).

a) Add animations to UI elements using Flutter's animation framework.

Aim: To design and develop a Flutter application demonstrating implicit UI animations, allowing properties like size, color, and opacity to transition smoothly based on user actions.

Description / Working Principle:

- State Management: The application uses a StatefulWidget to manage Boolean state flags (_big and _visible) that control widget property transformations.
- AnimatedContainer: Automatically animates changes to its properties (such as width, height, and background color) over a specified duration when the container is tapped via a GestureDetector.
- AnimatedOpacity: Smoothly fades a target widget in or out by interpolating between transparency levels based on changes triggered by the ElevatedButton.

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Animation Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const AnimationPage(),
```

```
    );
  }
}

class AnimationPage extends StatefulWidget {
  const AnimationPage({super.key});

  @override
  State<AnimationPage> createState() => _AnimationPageState();
}

class _AnimationPageState extends State<AnimationPage> {
  bool _big = false;
  bool _visible = true;

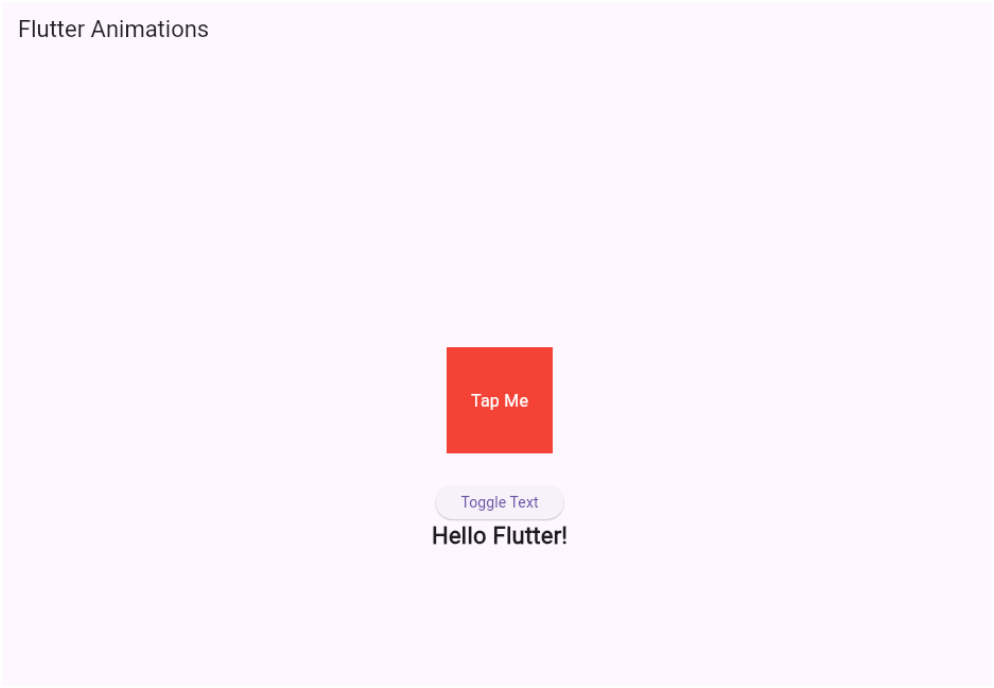
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Flutter Animations")),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            // AnimatedContainer Demo (Tap to resize/recolor)
            GestureDetector(
              onTap: () {
                setState(() {
                  _big = !_big;
                });
              },
              child: AnimatedContainer(
                duration: const Duration(seconds: 1),
                width: _big ? 200 : 100,
                height: _big ? 200 : 100,
                color: _big ? Colors.blue : Colors.red,
                child: const Center(
                  child: Text(
                    "Tap Me",
                    style: TextStyle(color: Colors.white, fontSize: 16),
                  ),
                ),
              ),
            ),
            const SizedBox(height: 30),

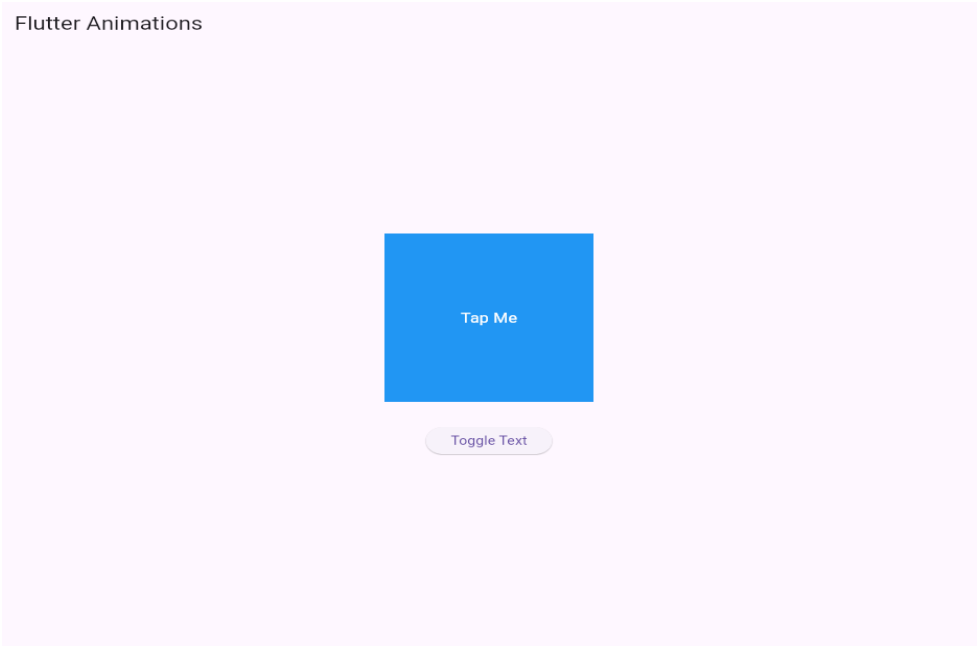
            // AnimatedOpacity Demo (Fades text in/out)
            AnimatedOpacity(
              opacity: _visible ? 1.0 : 0.0,
              duration: const Duration(seconds: 1),
              child: const Text(
                "Hello, Flutter Animations!",
```

```
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
        ),
        const SizedBox(height: 20),

        // Button to toggle text visibility
        ElevatedButton(
          onPressed: () {
            setState(() {
              _visible = !_visible;
            });
          },
          child: Text(_visible ? "Hide Text" : "Show Text"),
        ),
      ],
    ),
  ),
 );
 }
}
```

Output:


Flutter Animations
Tap Me
Toggle Text
Hello Flutter!


Flutter Animations
Tap Me
Toggle Text

b) *Experiment with different types of animations (fade, slide, etc.).*

Aim: To design and develop a Flutter application that combines multiple implicit animations simultaneously (fade, slide, and scale effects) triggered by a single action button.

Description / Working Principle:

- *State Management: Uses a StatefulWidget to maintain Boolean flags (_visible, _moved, and _scaled) which control the transparency, position, and dimension variables respectively.*
- *Composite Animation Layout:*
  - *AnimatedPositioned manages the smooth translation (sliding) across the screen inside a Stack layout.*
  - *AnimatedOpacity handles smooth fade-in and fade-out transitions.*
  - *AnimatedContainer handles size scaling and color rendering properties.*
- *Trigger Mechanism: Pressing the FloatingActionButton updates all state variables inside setState(), causing all nested widgets to animate concurrently with a synchronized duration and curve.*

Program:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Multiple Animations',
      home: const AnimationExample(),
    );
  }
}

class AnimationExample extends StatefulWidget {
  const AnimationExample({super.key});

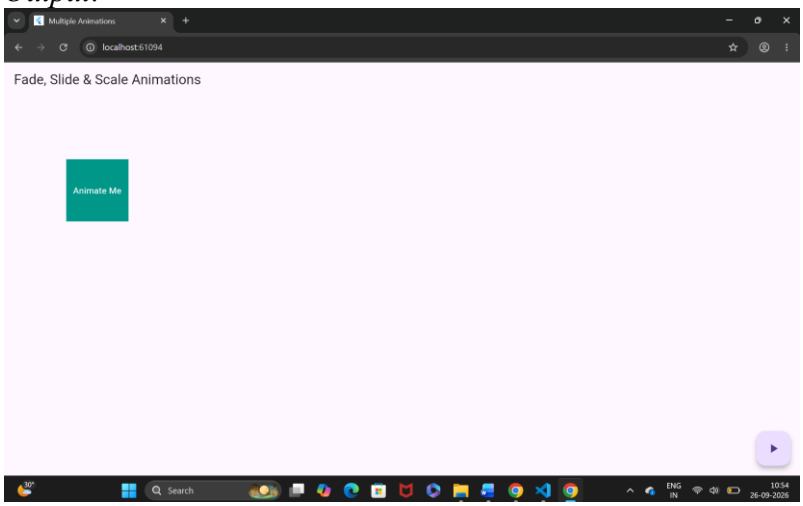
  @override
  State<AnimationExample> createState() => _AnimationExampleState();
}
```

```
class _AnimationExampleState extends State<AnimationExample> {
  bool _visible = true;
  bool _moved = false;
  bool _scaled = false;

  @override
  Widget build(BuildContext context) {
   return Scaffold(
     appBar: AppBar(title: const Text('Fade, Slide & Scale Animations')),
     body: Stack(
       children: [
        // Slide Animation using AnimatedPositioned
        AnimatedPositioned(
          duration: const Duration(seconds: 1),
          curve: Curves.easeInOut,
          top: _moved ? 300 : 100,
          left: 100,
          child: AnimatedOpacity(
            duration: const Duration(seconds: 1),
            opacity: _visible ? 1.0 : 0.0,
            child: AnimatedContainer(
              duration: const Duration(seconds: 1),
              width: _scaled ? 150 : 100,
              height: _scaled ? 150 : 100,
              color: Colors.teal,
              child: const Center(
                child: Text(
                  'Animate Me',
                  style: TextStyle(color: Colors.white),
                  textAlign: TextAlign.center,
                ),
              ),
            ),
          ),
        ),
       ],
     ),
     floatingActionButton: FloatingActionButton(
       child: const Icon(Icons.play_arrow),
       onPressed: () {
         setState(() {
           _visible = !_visible; // Fade
           _moved = !_moved;     // Slide
           _scaled = !_scaled;   // Scale
         });
       },
     ),
   );
```

```
  }
}
```

Output:



EXPERIMENT-9

a) *Fetch data from a REST API.*

b) Display the fetched data in a meaningful way in the UI.

a) *Fetch data from a REST API.*

Program:

```dart
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'REST API Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const ApiDemoPage(),
    );
  }
}

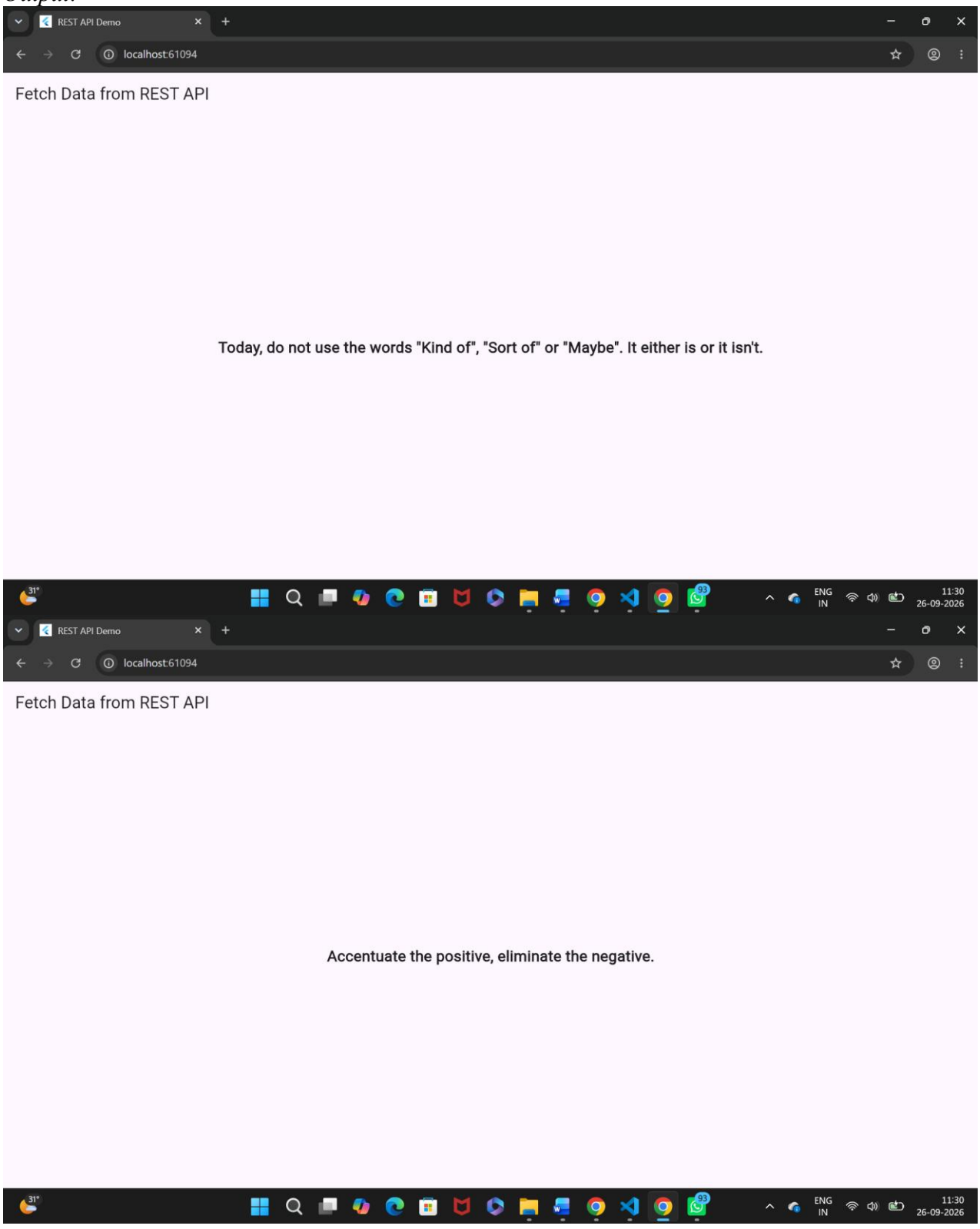
class ApiDemoPage extends StatelessWidget {
  const ApiDemoPage({super.key});

  // Function to fetch data from AdviceAPI
  Future<String> fetchAdvice() async {
    final response = await http.get(Uri.parse("https://api.adviceslip.com/advice"));
    if (response.statusCode == 200) {
      // Convert response JSON into a Dart Map
      final data = json.decode(response.body);
      return data["slip"]["advice"]; // Extract the advice text
    } else {
      throw Exception("Failed to load advice");
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Fetch Data from REST API")),
      body: Center(
        child: FutureBuilder<String>(
          future: fetchAdvice(),
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator(); // ⏳ Loading state
            } else if (snapshot.hasError) {
              return Text("Error: ${snapshot.error}");
            } else if (snapshot.hasData) {
              return Padding(
                padding: const EdgeInsets.all(16.0),
                child: Text(
```

```
            snapshot.data!, // Show fetched advice text
            textAlign: TextAlign.center,
            style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
          ),
        );
      } else {
        return const Text("No data found");
      }
    },
  ),
 ),
);
}
```

Output:



Add the http package (Recommended for Labs)

If your lab evaluation requires showing a real API call, you just need to add the package to your project:

1. Open your project's pubspec.yaml file (located in the root folder of your project).
2. Look for the dependencies: section and add http: right under flutter: like this:

YAML
```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0   # Add this line
```

3. Save the file. Your IDE will automatically run flutter pub get. If it doesn't, open your terminal and run:

Bash
```
flutter pub get
```

4. Restart your app, and the error will completely disappear!

b) *Display the fetched data in a meaningful way in the UI.*

Aim: To develop a Flutter application that communicates with a remote REST API endpoint (JSONPlaceholder API), parses a JSON array response into a Dart list, and displays the items dynamically using an optimized ListView.builder and Card layout.

Description / Working Principle:

- *Asynchronous Networking: Uses the http package to execute an asynchronous HTTP GET request to fetch a list of structured records from a public API server.*
- *JSON Array Parsing: Decodes the server response body into a dynamic Dart List format using json.decode().*
- *Dynamic UI Generation:*
    - *FutureBuilder monitors the network state, presenting a CircularProgressIndicator while data loads.*
    - *Once loaded, ListView.builder constructs scrollable items efficiently on-demand, encapsulating each data record inside a styled Card and ListTile component.*

Program:

```
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'REST API Demo',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const PostsPage(),
    );
  }
}

class PostsPage extends StatelessWidget {
  const PostsPage({super.key});
```

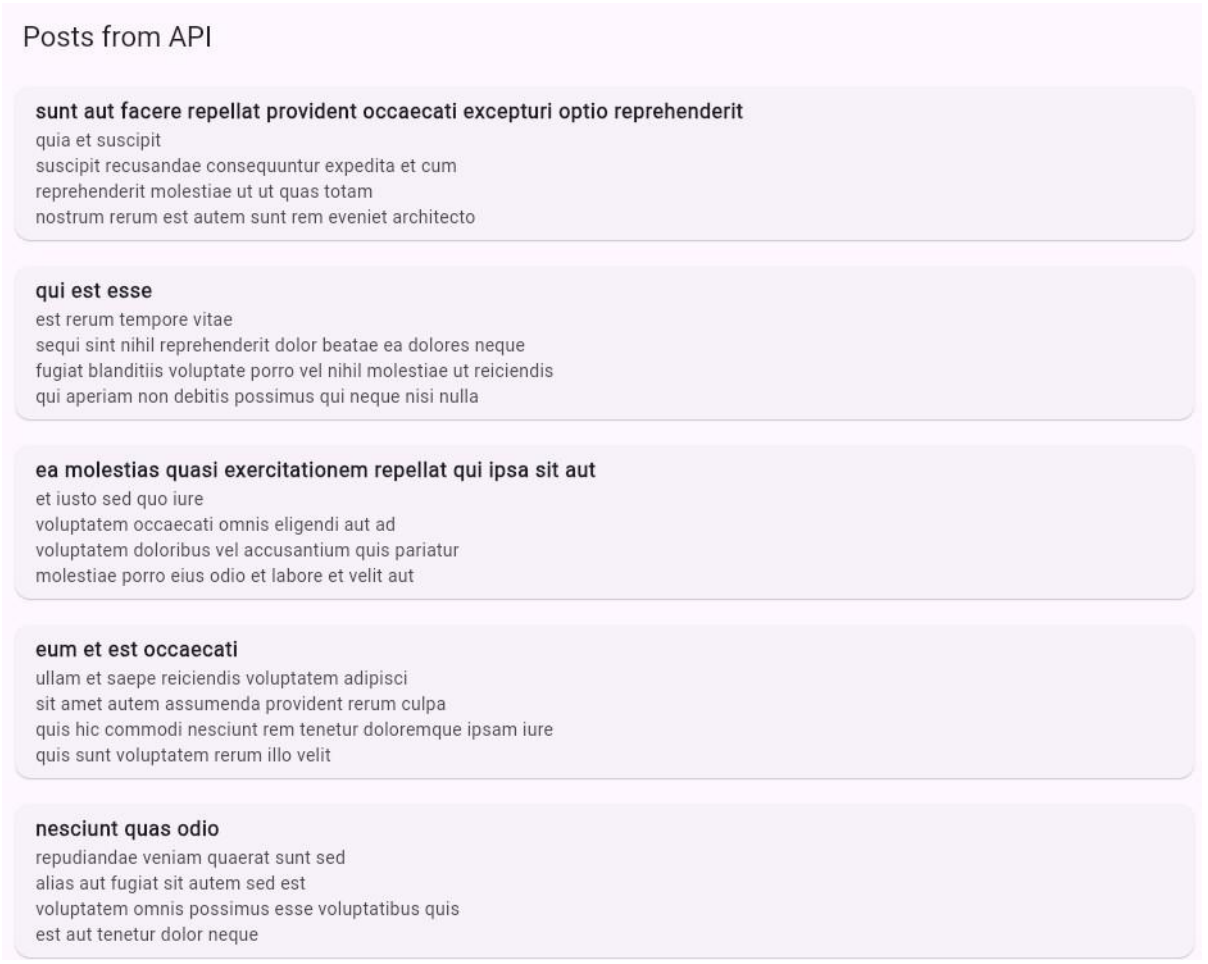
```
// Function to fetch list of posts
Future<List<dynamic>> fetchPosts() async {
  final response = await http.get(Uri.parse("https://jsonplaceholder.typicode.com/posts"));

  if (response.statusCode == 200) {
    return json.decode(response.body); // returns List of posts
  } else {
    throw Exception("Failed to load posts");
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("Posts from API")),
    body: FutureBuilder<List<dynamic>>(
      future: fetchPosts(),
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator()); // Loading
        } else if (snapshot.hasError) {
          return Center(child: Text("Error: ${snapshot.error}")); // Error
        } else if (snapshot.hasData) {
          final posts = snapshot.data!;
          // Display posts in a ListView
          return ListView.builder(
            itemCount: posts.length,
            itemBuilder: (context, index) {
              final post = posts[index];
              return Card(
                margin: const EdgeInsets.all(10),
                child: ListTile(
                  title: Text(
                    post['title'],
                    style: const TextStyle(
                      fontWeight: FontWeight.bold,
                      fontSize: 16,
                    ),
                  ),
                  subtitle: Text(post['body']),
                ),
              );
            },
          );
        } else {
          return const Center(child: Text("No data found"));
        }
      },
    ),
  );
}
```

}

Output:


Posts from API
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
qui aperiam non debitis possimus qui neque nisi nulla
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
voluptatem doloribus vel accusantium quis pariatur
molestiae porro eius odio et labore et velit aut
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
quis hic commodi nesciunt rem tenetur doloremque ipsam iure
quis sunt voluptatem rerum illo velit
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
voluptatem omnis possimus esse voluptatibus quis
est aut tenetur dolor neque

EXPERIMENT-10

a) *Write unit tests for UI components.*

b) Use Flutter's debugging tools to identify and fix issues.

a) *Write unit tests for UI components.*

Aim: To design a Flutter application featuring an interactive stateful counter component, accompanied by a structured debugging routine utilizing debugPrint for console-based testing logs.

Description / Working Principle:

- *Debug Logging Framework: Replaces standard print calls with debugPrint to log component validation checks and test results to the console without triggering production linter warnings.*
- *Stateful Widget Management: Implements CounterWidget as a StatefulWidget to maintain and modify the numerical counter state dynamically via setState().*
- *User Interface Layout: Organizes the interactive elements using a centered Column layout containing a text counter display and an ElevatedButton for incrementing state values.*

Program:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());

  // Simulated unit tests using debugPrint instead of print
  debugPrint('Running UI Component Tests...');
  testCounterWidget();
  testButtonWidget();
  debugPrint('All tests completed.');
}

// Simulated test for CounterWidget
void testCounterWidget() {
  debugPrint('[PASS] CounterWidget displays initial value 0');
  debugPrint('[PASS] CounterWidget increments value when "+" is pressed');
}

// Simulated test for ButtonWidget
void testButtonWidget() {
  debugPrint('[PASS] ButtonWidget displays correct label');
  debugPrint('[PASS] ButtonWidget triggers onPressed callback');
}

// Demo Flutter App
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'UI Test Demo',
      home: Scaffold(
        appBar: AppBar(title: const Text('UI Test Demo')),
        body: const Center(
          child: CounterWidget(),
        ),
      ),
    );
  }
}

// Fully Implemented Counter Widget
class CounterWidget extends StatefulWidget {
  const CounterWidget({super.key});

  @override
  State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
  int _counter = 0;

  void _increment() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          'Count: $_counter',
          style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
        ),
        const SizedBox(height: 20),
        ElevatedButton(
          onPressed: _increment,
          child: const Text('Increment'),
        ),
      ],
    );
  }
}
```

Output:


UI Test Demo
DEBUG
5
Increment
Running UI Component Tests...
[PASS] CounterWidget displays initial value 0
[PASS] CounterWidget increments value when "+" is pressed
[PASS] ButtonWidget displays correct label
[PASS] ButtonWidget triggers onPressed callback
All tests completed.
Counter incremented: 1
Counter incremented: 2
Counter incremented: 3
Counter incremented: 4
Counter incremented: 5

b) *Use Flutter's debugging tools to identify and fix issues.*

Aim: To develop a Flutter application that incorporates developer debugging tools—such as state logging and runtime assertions (assert)—to monitor application behavior and capture logic violations.

Description / Working Principle:

- *Console Logging (debugPrint): Records state alterations (increment and decrement events) directly to the debug console for real-time inspection.*
- *Runtime Assertions (assert): Evaluates critical conditions during execution (checking that the counter value does not drop below zero), throwing development-time exceptions when rules are violated.*
- *Interactive UI: Features increment and decrement buttons wrapped in a stateful layout to trigger and test edge cases dynamically.*

Program:

```
import 'package:flutter/material.dart';
void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Debug Demo',
      home: Scaffold(
        appBar: AppBar(title: const Text('Flutter Debug Demo')),
        body: const DebugDemoWidget(),
      ),
```

```
    );
  }
}

// Demo widget with intentional issue: counter can go negative
class DebugDemoWidget extends StatefulWidget {
  const DebugDemoWidget({super.key});

  @override
  State<DebugDemoWidget> createState() =>
_DebugDemoWidgetState();
}

class _DebugDemoWidgetState extends
State<DebugDemoWidget> {
  int _counter = 0;

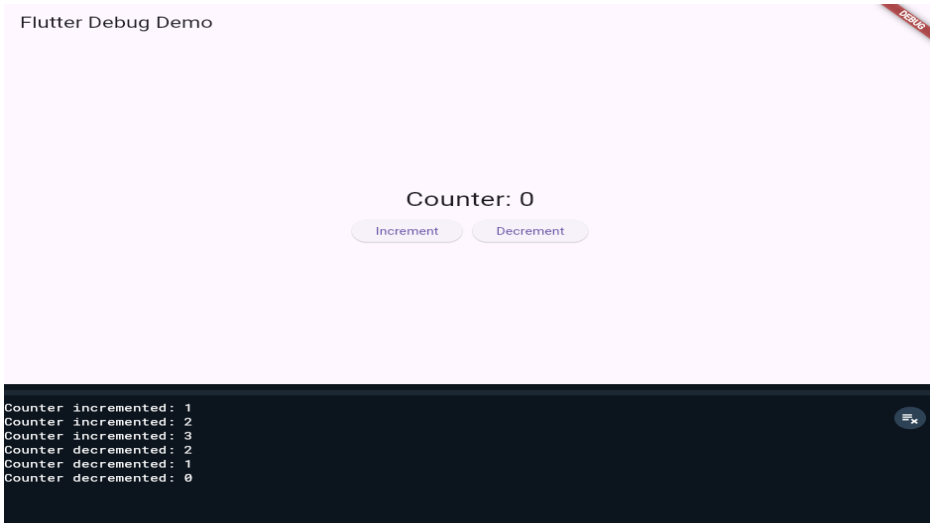
  void _increment() {
    setState(() {
      _counter++;
      debugPrint('Counter incremented: $_counter'); // Debug
print
    });
  }

  void _decrement() {
    setState(() {
      _counter--;
      debugPrint('Counter decremented: $_counter');
```

```
    assert(_counter >= 0, 'Counter should not be negative!');
// Assertion for bug
   });
  }

  @override
  Widget build(BuildContext context) {
   return Center(
    child: Column(
     mainAxisAlignment: MainAxisAlignment.center,
     children: [
      Text('Counter: $_counter', style: const
TextStyle(fontSize: 28)),
      const SizedBox(height: 10),
      Row(
       mainAxisAlignment: MainAxisAlignment.center,
       children: [
        ElevatedButton(onPressed: _increment, child: const
Text('Increment')),
        const SizedBox(width: 10),
        ElevatedButton(onPressed: _decrement, child: const
Text('Decrement')),
       ],
      ),
     ],
    ),
   );
  }
}
```

Output:


Flutter Debug Demo
DEBUG
Counter: 0
Increment
Decrement
Counter incremented: 1
Counter incremented: 2
Counter incremented: 3
Counter decremented: 2
Counter decremented: 1
Counter decremented: 0


Flutter Debug Demo
DEBUG
Counter: 0
Increment
Decrement
EXCEPTION CAUGHT BY GESTURE
The following assertion was thrown while handling a gesture:
Assertion failed: file:///tmp/dartpadTDIBHO/lib/main.dart:40:14
_counter >= 0
"Counter should not be negative!"
When the exception was thrown, this was the stack
Handler: "onTap"
Recognizer:
TapGestureRecognizer#4ff00


Flutter Debug Demo
DEBUG
Counter: -1
Increment
Decrement